

CS3231: Theory of Computation

This course examines fundamental aspects of computation that every computer scientist should know. What is a finite automaton and how does it relate to regular expressions (and searching a database)? What is a context-free language and how does it relate to parsing languages? What is the P vs. NP problem and why does it matter? How do we decide if a problem is easy or hard? This course introduces techniques for precisely formulating problems and studying their properties using a variety of commonly used reasoning techniques (e.g., model equivalence, non-determinism, digitalisation, simulation and reduction).

Contents

1	Prerequisites and Definitions	2
2	Regular Languages	3
2.1	Deterministic Finite Automata	3
2.2	Nondeterministic Finite Automata	4
2.3	DFAs vs NFAs	6
2.4	Minimisation of Automata	7
2.5	Parallel Simulation of n-DFAs	8
2.6	Regular Expressions	9
2.7	Regular Languages	12
2.8	Linear Grammars	14
3	Context Free Languages	15
3.1	Context-Free Grammars	15
3.2	Ambiguous Grammars	18
3.3	Simplification of CFG	18
3.4	Pushdown Automata	21
3.5	CFGs vs PDAs	24
3.6	Context Free Languages	26
3.7	Deterministic PDA	30
4	Turing Machines	31
4.1	Turing Machines	32
4.2	Languages of a TM	33
4.3	Undecidability	35
4.4	Undecidable Problems	37
4.5	Unrestricted Grammars	39
5	Complexities and Classes	40
5.1	Definitions	41
5.2	NP-Completeness	41
5.3	Known NP-Complete Problems	42
5.4	Modifications to TMs	44
5.5	Complexity Classes	44

1 Prerequisites and Definitions

Definition (Alphabet). *Alphabets* are finite non-empty sets of symbols, denoted Σ . Ambiguous alphabets like $\{a, ab, b\}$ are not used.

Definition (String). *Strings* are finite sequences of symbols chosen from a given alphabet.

Notation. We denote empty strings as ϵ , and the length (number of symbols) of string $|\omega|$. For strings x and y , xy denotes the *concatenation* of x and y .

Example (Powers of an Alphabet). We define Σ^k to be the set of all strings of length k over the alphabet. For alphabet $\Sigma = \{0, 1\}$,

$$\Sigma^0 = \{\epsilon\}, \quad \Sigma^1 = \{0, 1\}, \quad \Sigma^2 = \{00, 01, 10, 11\}, \quad \Sigma^{\leq 2} = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2.$$

In addition, we have the set of all strings given an alphabet

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

To exclude the empty string, we denote Σ^+ . $\Sigma^* = \Sigma^+ \cup \{\epsilon\}$.

Definition (Language). *Languages* are sets of strings all of which are chosen from some Σ^* where Σ is a particular alphabet. If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over Σ .

Notation. We denote $L_1 \cdot L_2 = L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$. For language L ,

$$\begin{aligned} L^* &= \{x_1 x_2 \dots x_n : x_1, x_2, \dots, x_n \in L, n \in \mathbb{N}\} = \{\epsilon\} \cup L \cup LL \cup \dots \\ L^+ &= \{x_1 x_2 \dots x_n : x_1, x_2, \dots, x_n \in L, n \in \mathbb{N}^*\} = L \cup LL \cup LLL \cup \dots \\ L^+ &= LL^*. \quad qL^* = L^+ \iff \epsilon \in L. \end{aligned}$$

Corollary. If L is a language over Σ , L is a language over any alphabet $K \supseteq \Sigma$.

Theorem. Number of strings over any fixed finite alphabet Σ is countable, as the union of countably many countable sets is countable.

Theorem. Number of languages over any non-empty alphabet is uncountable.

Proof. Consider the alphabet $\Sigma = \{a\}$, and the binary representation of any real number in $[0, 1) : 0.b_0 b_1 b_2 \dots$. Let $L = \{a^i : b_i = 1\}$. L corresponds to each $x \in [0, 1)$. Hence the number of languages is at least as large as the set of real numbers, which is uncountable. ■

Definition (Reverse). The *reversal* of a string ω denoted ω^R is given by induction on the length of the string:

1. If $|\omega| = 0$, then $\omega^R = \omega = \epsilon$
2. If $|\omega| = n + 1 > 0$, then $\omega = ua$ for some $a \in \Sigma$, and $\omega^R = au^R$

We further define, for a language L , $L^R = \{\omega^R : \omega \in L\}$.

Proposition. For any strings ω and x , $(\omega x)^R = x^R \omega^R$

Proof.

Basis Step. $|x| = 0$. Then $x = \epsilon$, $(\omega x)^R = (\omega \epsilon)^R = \omega^R = \epsilon \omega^R = \epsilon^R \omega^R = x^R \omega^R$.

Induction Hypothesis. If $|x| \leq n$, then $(\omega x)^R = x^R \omega^R$.

Induction Step. Let $|x| = n + 1$. Then $x = ua$ for some $u \in \Sigma^*$ and $a \in \Sigma$ s.t. $|u| = n$.

$$\begin{aligned}
 (\omega x)^R &= (\omega (ua))^R \\
 &= ((\omega u) a)^R \\
 &= a(\omega u)^R \\
 &= a u^R \omega^R \\
 &= (ua)^R \omega^R \\
 &= x^R \omega^R \quad \blacksquare
 \end{aligned}$$

2 Regular Languages

We begin by consider a class of languages known as the **regular languages**. These languages can be defined by *finite automaton*, *regular expressions*, or a reduced form of recursive notation known as *linear grammars*.

Intuitively, regular languages comprises the finite languages and infinite languages that involve modulo counting.

2.1 Deterministic Finite Automata

Definition (DFA). A DFA $A = (Q, \Sigma, \delta, q_0, F)$, where:

- Q : A finite set of states
- Σ : A finite set of input symbols
- δ : A transition function taking as argument a state and an input symbol and returns a state. $\delta : Q \times \Sigma \rightarrow Q$
- q_0 : A start state, $q_0 \in Q$
- F : A set of final or accepting states. $F \in \mathcal{P}(Q)$

Remark. In CS3231, we consider only DFA with finite set of states.

Suppose $a_1 a_2 \cdots a_n$ a sequence of input symbols. Starting at the start state q_0 , we consult the transition function δ , to find the state of the DFA $\delta(q_0, a_1) = q_1$ after processing the first input symbol a_1 .

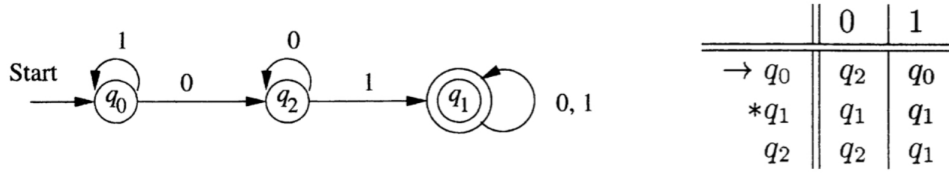
Example. To generate the language $L = \{\omega \mid \omega \text{ is of the form } x01y \text{ for some strings } x \text{ and } y \text{ consisting of 0's and 1's only}\}$, consider the DFA $A = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\})$, where δ is defined:

$$\delta(q_0, 0) = q_2, \quad \delta(q_0, 1) = q_0, \quad \delta(q_1, 0) = \delta(q_1, 1) = q_1, \quad \delta(q_2, 0) = q_2, \quad \delta(q_2, 1) = q_1$$

We have alternative notations for description of DFAs, namely the transition diagram and the transition table.

To extend the transition function to strings, we define an **extended transition function** $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ to describe the state (given a starting state), after processing a sequence of input symbols. We define by induction on the length of the input string:

1. $\hat{\delta}(q, \epsilon) = q$. Starting in state q , if no inputs are read, no change in state.



2. Suppose w a string of the form xa , where $x \in \Sigma^*$, $a \in \Sigma$,

$$\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a).$$

That is, we first process all but the last symbol of w , then its last symbol.

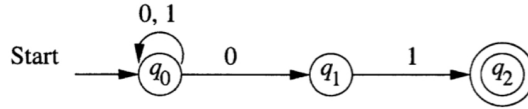
Definition (Language of a DFA). The *language* of a DFA A is denoted $L(A)$, and is defined by

$$L(A) = \{\omega \mid \hat{\delta}(q_0, \omega) \in F\},$$

the set of strings ω that the start state q_0 may take to arrive at one of the accepting states.

2.2 Nondeterministic Finite Automata

We now consider an automata in which we can be in several state at once. In an NFA, it is not necessary that all states have all inputs symbols defining a transition. In those situations, the thread of the NFA's existence corresponding to those states simply "dies".



Definition (NFA). An *NFA* $A = (Q, \Sigma, \delta, q_0, F)$, where:

- Q : A finite set of states
- Σ : A finite set of input symbols
- δ : A transition function taking as argument a state and an input symbol and returns **a set of states**.
 $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$
- q_0 : A start state, $q_0 \in Q$
- F : A set of final or accepting states. $F \in \mathcal{P}(Q)$

Remark. The only difference between an NFA and a DFA is the return type of the transition function.

Similarly, we define the **extended transition function** $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$, the set of states that the NFA will be in if it starts in a given state q and processes the string ω . We define by induction on the length of the input string:

1. $\hat{\delta}(q, \epsilon) = \{q\}$. If no inputs are read, we are only in the state we began in.
2. Suppose w a string of the form xa , where $x \in \Sigma^*$, $a \in \Sigma$, and that $\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$

$$\hat{\delta}(q, w) = \bigcup_{i=1}^k \delta(p_i, a) = \{r_1, r_2, \dots, r_m\}$$

That is, we first compute $\hat{\delta}(q, x)$, then follow any transition from any of the possible states.

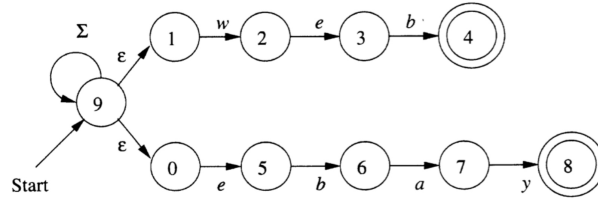
Definition (Language of an NFA). The *language* of an NFA A is denoted $L(A)$, and is defined by

$$L(A) = \left\{ \omega \mid \hat{\delta}(q_0, \omega) \cap F \neq \emptyset \right\},$$

the set of string ω that the start state may take that does not definitively lead to a nonaccepting state. The set of strings such that the possibilities contain at least one accepting state.

NFAs with ϵ -Transitions

We consider an automaton in which we allow a transition on ϵ . The NFA is hence allowed to make a transition without receiving an input symbol.



Definition (ϵ -NFA). An ϵ -NFA $A = (Q, \Sigma, \delta, q_0, F)$, where:

- Q : A finite set of states
- Σ : A finite set of input symbols
- δ : A transition function taking as argument a state and an input symbol (including ϵ) and returns **a set of states**. $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$
- q_0 : A start state, $q_0 \in Q$
- F : A set of final or accepting states. $F \in \mathcal{P}(Q)$

To define an extended transition function for ϵ -NFAs, we first define the ϵ -closure of a state recursively:

1. $q \in \epsilon\text{-closure}(q)$
2. If $p \in \epsilon\text{-closure}(q) \wedge \delta(p, \epsilon) = r \implies r \in \epsilon\text{-closure}(q)$.

The ϵ -closure of a state q is the set of all states reachable from q using only ϵ transitions. We further define the ϵ -closure of a set of states S :

$$\epsilon\text{-closure}(S) = \bigcup_{q \in S} \epsilon\text{-closure}(q).$$

We now define the **extended transition function** $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$, the set of states that the ϵ -NFA will be in if it starts in a given state q and processes the string ω . We define by induction on the length of the input string:

1. $\hat{\delta}(q, \epsilon) = \epsilon\text{-closure}(q)$. (by the definition of ϵ -closure)
2. Suppose w a string of the form xa , where $x \in \Sigma^*$, $a \in \Sigma$ (WLOG, ϵ not in Σ), and that $\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$

$$\hat{\delta}(q, w) = \bigcup_{i=1}^k \delta(p_i, a) = \epsilon\text{-closure}(\{r_1, r_2, \dots, r_m\})$$

That is, we first compute $\hat{\delta}(q, x)$, then follow any transition from any of the possible states, then consider the possibility of any ϵ transitions from the possible states.

Remark. A ϵ -NFA $E = (Q_E, \Sigma, \delta_E, q_0, F_E)$ accepts the same language as the DFA $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$ where:

- $Q_D = \{\epsilon\text{-closure}(S) \mid S \in \mathcal{P}(Q_E)\}$
- $q_D = \epsilon\text{-closure}(q_0)$
- $F_D = \{S \mid S \subseteq Q_D \wedge S \cap F_E \neq \emptyset\}$
- $\delta_D(S, a) = \epsilon\text{-closure}\left(\bigcup_{p \in S} \delta_E(p, a)\right)$

2.3 DFAs vs NFAs

Equivalence of DFAs and NFAs

Theorem. A language L is accepted by some DFA if L is accepted by some NFA.

Proof. Suppose an NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$. We consider the DFA $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$, where

- $Q_D = \mathcal{P}(Q_N)$. Intuitively, each state of the DFA represents a state of the entire NFA.

Remark. This conversion is known as *subset construction*, involving constructing all subsets of the set of states of the NFA. Often, not all states are accessible from the start state. Inaccessible states can be discarded.

- $F_D = \{S \mid S \subseteq Q_N \wedge S \cap F_N \neq \emptyset\} \subseteq \mathcal{P}(Q_N)$. That is, F_D consists all sets of N 's states that include at least one accepting state of N .
- $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$, given $S \in \mathcal{P}(Q_N)$, and $a \in \Sigma$. We consider all possible current states, and take the union of all possible states after processing input a

To prove that $\forall w \in \Sigma^* \left(\hat{\delta}_D(\{q_0\}, w) = \hat{\delta}_N(q_0, w) \right)$, we induct on the length of w :

1. *Base case:* $w = \epsilon$

$$\hat{\delta}_D(\{q_0\}, \epsilon) = \{q_0\} = \hat{\delta}_N(q_0, \epsilon).$$

2. *Inductive step:*

$$\begin{aligned} \hat{\delta}_D(\{q_0\}, wa) &= \delta_D(\hat{\delta}_D(\{q_0\}, w), a) \\ &= \bigcup_{q \in \hat{\delta}_D(\{q_0\}, w)} \delta_N(q, a) \\ &= \bigcup_{q \in \hat{\delta}_N(q_0, w)} \delta_N(q, a) \\ &= \hat{\delta}_N(q_0, wa). \end{aligned}$$

Since $\hat{\delta}_D(\{q_0\}, w) \in F_D \iff \hat{\delta}_N(q_0, wa) \cap F_N \neq \emptyset$, we have $L(N) = L(D)$. ■

Theorem. A language L is accepted by some DFA only if L is accepted by some NFA.

Proof. Intuitively, we take the transition diagram for the DFA as a transition diagram for an NFA, which has only one choice of transition at any state. ■

Consider a language that can be described by some NFA and some DFA. The DFA in practice has about as many states as the NFA, but often with more transitions. In the worst case, however, the smallest DFA can have 2^n states while the smallest NFA for the same language only has n states.

Languages of Automata

Definition (Regular Language). If L is $L(A)$ for some DFA A , then L is a regular language.

Corollary. L is a regular language $\iff L$ is $L(N)$ for some NFA N .

Theorem. Given a non-empty finite alphabet Σ , there exists a countable number of regular languages.

Since each regular language can be represented by an automaton, there exist finite, or countably infinite regular languages. That is, there exist languages that are not accepted by any automata and is hence non-regular.

2.4 Minimisation of Automata

Prerequisites

Suppose a regular language L . We define the equivalence relation for $u, w \in \Sigma^*$

$$u \equiv_L w \iff [\forall x \in \Sigma, ux \in L \iff wx \in L].$$

We denote $\text{equiv}(w)$ the equivalence class of w .

Definition (Distinguishable States). We say that (p, q) is distinguishable $\iff \exists w \in \Sigma^*$ such that

$$\hat{\delta}(p, w) \in F \wedge \hat{\delta}(q, w) \notin F \quad \vee \quad \hat{\delta}(p, w) \notin F \wedge \hat{\delta}(q, w) \in F.$$

Remark.

$$(p, q) \text{ is indistinguishable } \iff [\forall w \in \Sigma^*, \hat{\delta}(p, w) \notin F \iff \hat{\delta}(q, w) \notin F].$$

Minimal DFA for L

If there is a finite number of equivalence classes, $\exists$ a DFA $D = (Q, \Sigma, \delta, q_0, F)$ where:

- $Q = \{\text{equiv}(\omega) \mid \omega \in \Sigma^*\} = \Sigma^* / \equiv_L$
- $\delta : (\Sigma^* / \equiv_L \times \Sigma) \rightarrow \Sigma^* / \equiv_L$ defined $\delta(\text{equiv}(\omega), a) = \text{equiv}(wa)$
- $q_0 = \text{equiv}(\epsilon)$
- $F = \{\text{equiv}(\omega) \mid \omega \in L\}$

which, by definition of the equivalence relation, accepts L . It is also the minimal DFA for L .

Lemma. $u \not\equiv_L \omega \implies \hat{\delta}'(q'_0, u) \neq \hat{\delta}'(q'_0, \omega)$.

Proof. Suppose not. That is $u \not\equiv_L \omega \wedge \hat{\delta}'(q'_0, u) = \hat{\delta}'(q'_0, \omega)$. Then $\exists x \in \Sigma^*$ such that WLOG, $ux \in L \wedge \omega x \notin L$. But $\hat{\delta}'(q'_0, u) = \hat{\delta}'(q'_0, \omega)$ and A' accepts either both or neither. Hence a contradiction. ■

Proposition. D is minimal.

Proof. Suppose a DFA $A' = (Q', \Sigma, \delta', q'_0, F')$ also accepting L . From the lemma, we have

$$u \not\equiv_L \omega \implies \hat{\delta}'(q'_0, u) \neq \hat{\delta}'(q'_0, \omega).$$

Let $u \equiv_{A'} w \iff \hat{\delta}'(q'_0, u) = \hat{\delta}'(q'_0, w)$. Then $u \equiv_L \implies u \equiv_{A'} w$. Hence $\equiv_{A'}$ divides the equivalence classes of $\equiv_L$ into finer equivalence classes, and $\equiv_L$ is minimal. ■

Minimal DFA Construction Algorithm

Suppose we are given a DFA $A = (Q, \Sigma, \delta, q_0, F)$. We determine all distinguishable pairs with the following recursive algorithm:

1. *Base case:* Each pair (p, q) such that $p \in F$ and $q \notin F$ (or vice versa) is distinguishable.
2. *Inductive step:* If $\exists a \in \Sigma [\delta(p, a) \text{ and } \delta(q, a) \text{ are distinguishable}]$, then (p, q) is distinguishable.
3. Steps 1 and 2 are repeated until no more pairs can be added.
4. The remaining pairs are non-distinguishable states.

To compute the distinguishable states, we utilise the table-filling algorithm: We then form the new DFA as

B	x						
C	x	x					
D	x	x	x				
E		x	x	x			
F	x	x	x		x		
G	x	x	x	x	x	x	
H	x		x	x	x	x	x
	A	B	C	D	E	F	G

follows:

1. Remove all unreachable states.
2. Find all nondistinguishable pairs of states using the algorithm. They are equivalent. This yields an equivalence relation
3. Form the new DFA with:
 - Q_{new} as the set of the equivalence classes.
 - The same Σ
 - The transitions from each equivalence class is based on the corresponding transition in the original DFA:

$$\delta(p, a) = q \implies \delta_{new}(E_p, a) = E_q.$$

where E_p, E_q are the equivalence classes of p, q respectively.

- $q_{0,new} = equiv(q_0)$
- $F_{new} = \{equiv(\omega) | \omega \in F\}$

2.5 Parallel Simulation of n-DFA's

Suppose two DFA's $A = (Q, \Sigma, \delta, q_0, F)$ and $A' = (Q', \Sigma, \delta', q'_0, F')$. We may construct another DFA $A'' = (Q \times Q', \Sigma, \delta'', (q_0, q'_0), F'')$, where

$$\delta''((q, q'), a) = (\delta(q, a), \delta'(q', a)).$$

and F'' depends on need.

Example (Definitions of F'').

- $F'' = F \times F'$ finds the intersections of the two automata.
- $F'' = F \times Q' \cup Q \times F'$ finds the union of the two automata.

This simulates the two DFAs in parallel.

Remark. By repeated application, we may simulate any boolean combination of automata.

2.6 Regular Expressions

Through regular expressions, another type of language-defining notation that algebraically describes a language, we are able to declaratively express the strings we want to accept. Each regular expression denotes a (regular) language.

Operators

Given a regular expression E , we denote $L(E)$ for the language that E denotes. We then define regular expressions inductively (with various operators), as follows:

1. *Basis Step:*

- Constant ϵ is a regular expression, with $L(\epsilon) = \{\epsilon\}$.
- Constant $\emptyset$ is a regular expression, with $L(\emptyset) = \emptyset$.
- If $a \in \Sigma$, a is a regular expression, with $L(a) = \{a\}$.

2. *Inductive Step:* Given regular expressions E and F ,

- $E + F$ is a regular expression denoting union, $L(E + F) = L(E) \cup L(F)$.
- EF is a regular expression denoting concatenation, $L(EF) = L(E)L(F)$.
- E^* is a regular expression denoting its (Kleene's) closure, $L(E^*) = (L(E))^*$
- E^+ is a regular expression denoting its closure, without ϵ , $L(E^+) = EE^*$
- (E) is a regular expression denoting the same language, $L((E)) = L(E)$

We further define the order of precedence of the operators:

1. The parentheses operator is of highest precedence, to specify the desired grouping of regular expressions.
2. The star operator is of highest precedence, applied to the smallest sequence of symbols to its left that is a well-formed regular expression.
3. The concatenation operator groups all expressions that are *juxtaposed* (adjacent without intervening operators). Concatenation is associative.
4. The union binary operator groups its operands. Union is associative.

Converting Automaton to Regular Expression

Theorem. If $L = L(A)$ for some DFA A , then there is a regular expression R such that $L = L(R)$. That is, every regular language can be defined by a regular expression.

Proof. Suppose a DFA $A = (Q, \Sigma, \delta, q_{start}, F)$. Without loss of generality, suppose $Q = \{1, 2, \dots, n\}$ for some n and $q_{start} = 1$. Let $R_{i,j}^k$ denote the regular expression for the set of strings which can be formed by going from state i to state j only using intermediate states $\leq k$. We have

$$L(A) = L\left(\sum_{j \in F} R_{1,j}^n\right)$$

We compute by induction:

1. *Base step:* $R_{i,j}^0$. That is, the path must have no intermediate states at all. We only have an arc from (state) i to j , or a path of length 0 consisting only i .

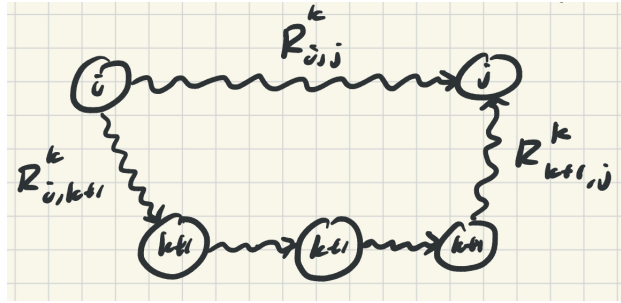
$$i \neq j \implies R_{i,j}^0 = a_1 + a_2 + \cdots + a_m, \quad \forall a_r \in \Sigma, \delta(i, a_r) = j.$$

$$i = j \implies R_{i,i}^0 = \epsilon + a_1 + a_2 + \cdots + a_m, \quad \forall a_r \in \Sigma, \delta(i, a_r) = i.$$

2. *Inductive step:* Suppose there is a path from state i to j going through states no higher than k .

$$R_{i,j}^{k+1} = R_{i,j}^k + R_{i,k+1}^k (R_{k+1,k+1}^k)^* R_{k+1,j}^k.$$

We then have the case where the path does not go through state $k+1$ at all, and the case where it goes through $k+1$ at least once.



We may then obtain $L(A) = L\left(\sum_{j \in F} R_{1,j}^n\right)$. ■

Remark. The construction of a regular expression from automata could be applied to NFA or ϵ -NFA, but we construct about n^3 expressions, with each's length growing by a factor of 4 each inductive step, resulting in an expression on the order of 4^n symbols for an n -state automaton.

Converting Regular Expression to Automaton

Theorem. Every language defined by a regular expression can be defined by an automaton.

We may construct an automaton through structural induction on the expression, constructing automata for the basis expressions and combining them into larger automata that fits the various operators.

Proof. Suppose $L = L(R)$ for a regular expression R . We construct a ϵ -NFA E with:

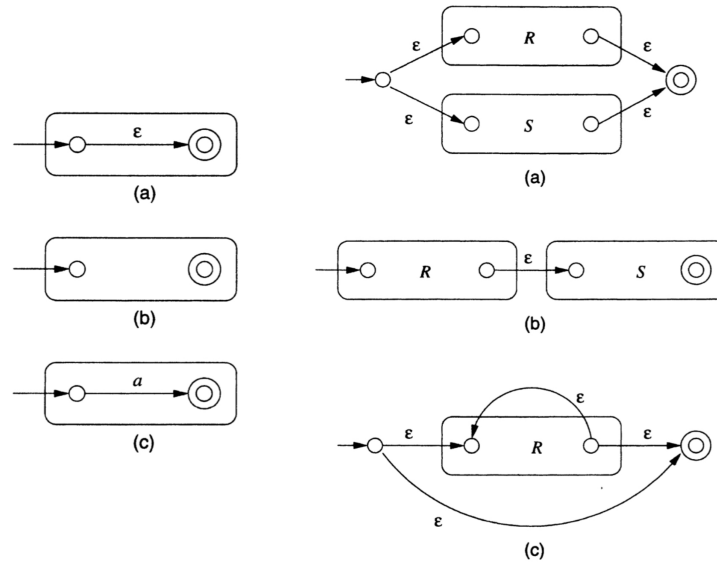
1. Exactly one accepting state.
2. No transition into the starting state.
3. No transition out of the accepting state.
4. Starting state $\neq$ Accepting state.

For the bases, we create automata that accepts the languages

- (a) $L(\epsilon)$,
- (b) $L(\emptyset)$,
- (c) $L(a)$

We then assume that we have the automaton for the immediate subexpressions of a given regular expression. We then have the four cases:

- (a) $R + S$,
- (b) RS ,
- (c) R^*
- R^+ which is constructed from the automaton of R^* , but without the ϵ transition from the starting to the accepting state
- (R) where the automaton does not change.



The constructed automata satisfy the four conditions, providing the language of the regular expression. ■

Algebra of Regular Expressions

We find various useful results for simplifying regular expressions and to keep the size of expressions manageable. We consider the algebra of regular expressions, analysing properties such as associativity, commutativity and distributivity, as well as relevant identities and annihilators. For regular expressions L, M, N :

Associativity, Commutativity and Distributivity

- $L + M = M + L$: Commutative law for union.
- $(L + M) + N = L + (M + N)$: Associative law for union. With the commutative law, we may take union in any order.
- $(LM)N = L(MN)$: Associative law for concatenation. Concatenation is non-commutative.
- $L(M + N) = LM + LN$: Left-distributive law of concatenation over union.
- $(M + N)L = ML + NL$: Right-distributive law of concatenation over union.

Identities, Annihilators and Idempotence

- $\emptyset + L = L + \emptyset = L$: $\emptyset$ identity for union.
- $\epsilon L = L\epsilon = L$: ϵ identity for concatenation.
- $\emptyset L = L\emptyset = \emptyset$: $\emptyset$ annihilator for concatenation.
- $L + L = L$: Idempotence law for union.

Closures

- $(L^*)^* = L^*$: Closing an expression that is already closed does not change.
- $\emptyset^* = \epsilon$
- $\epsilon^* = \epsilon$
- $L^+ = LL^* = L^*L$
- $L^* = L^+ + \epsilon$
- $(L + M)^* = (L^*M^*)^*$

2.7 Regular Languages

To show that a given language is nonregular, we may use the pumping lemma. Intuitively, we can always find a nonempty string y not too far from the beginning that can be repeated any number of times, or deleted while keeping the resulting string in L .

Lemma. *Let L be a regular language. Then $\exists n = f(L)$ such that $\forall \omega \in L, |\omega| \geq n, \exists x, y, z \in \Sigma^*, \omega = xyz$ such that:*

- $|xy| \leq n$. That is, string to be pumped is not too far from the start of the string.
- $y \neq \epsilon$. The string to be “pumped” cannot be empty.
- $\forall i \geq 0, xy^iz \in L$

The contrapositive of the lemma is used to prove that a language is nonregular. That is, $\forall n \in \mathbb{N}, \exists w \in L$, such that $|\omega| \geq n, \forall x, y, z \in \Sigma^*, \text{ where } \omega = xyz, |xy| \leq n, y \neq \epsilon$, we have

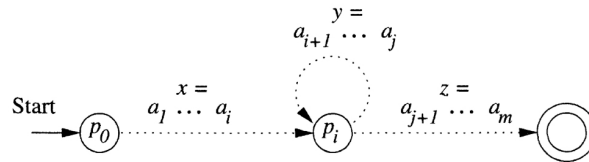
$$\exists i \in \mathbb{N} (xy^iz \notin L)$$

Proof. Suppose L is regular. Then $\exists$ DFA $A = (Q, \Sigma, \delta, q_0, F)$ such that $L = L(A)$. Suppose $|Q| = n$. Consider a string $\omega \in \Sigma^*, |\omega| \geq n$. Then $w = a_1a_2 \cdots a_m$ for some $m \geq n, a_i \in \Sigma$. For $i = 0, 1, \dots, n, p_i := \hat{\delta}(q_0, a_1a_2 \cdots a_i)$. p_i is the state of A after reading the first i symbols of ω .

Since there are n different states and $n + 1$ different p_i 's, by the pigeonhole principle, $\exists 0 \leq i < j < n$ such that $p_i = p_j$. Let:

- $x = a_1a_2 \cdots a_i$
- $y = a_{i+1}a_{i+2} \cdots a_j$
- $z = a_{j+1}a_{j+2} \cdots a_m$

By processing x , A takes up state p_i , after which y takes it from p_i back to p_i , and z is the balance of ω . $x = \epsilon \iff i = 0$, and $z = \epsilon \iff j = n = m$. However, y cannot be empty, since $i < j$. Then consider for



any $k \in \mathbb{N}$, the string xy^kz :

- If $k = 0$, the automaton goes from $q_0 = p_0$ to $p_i = p_j$ on input x , and then from $p_i = p_j$ to the accepting state on input z . A accepts xz .

- If $k > 0$, the automaton goes from q_0 to p_i on input x , and then circles p_i to p_i k times on input y^k , and then to the accepting state on input z . A accepts xy^kz . ■

By using the pumping lemma, we can construct a proof of the following structure to show a specified language is non-regular:

Proposition. $L = \{a^m b^m | m \geq 1\}$ is nonregular.

Proof. Suppose L is regular. Let n be as in the Pumping Lemma. Let $\omega = a^n b^n$. Let $\omega = xyz$ as in the pumping lemma. Then y consists of only a 's, and $|y| > 0$, $y = \{a\}^+$. By the pumping lemma, $xy^2z \in L$. But xy^2z contains more a 's than b 's, and $xy^2z \notin L$. Hence L is nonregular.

Proposition. $L = \{a^p | p \text{ is prime}\}$ is nonregular.

Proof. Suppose L is regular. Let n be as in the Pumping Lemma. Let $\omega = a^p$, where p is prime and $p > n$. Let $\omega = xyz$ as in the pumping lemma. By the pumping lemma, $\forall k \in \mathbb{N}, xy^kz \in L$. Consider $k = p + 1$. Then $xy^kz = a^r$ where

$$r = |x| + |z| + (p + 1)|y| = p + p|y|$$

and r is non-prime, and $xy^{p+1}z \notin L$. Hence L is nonregular.

Proposition. $L = \{a^i b^j | i < j\}$ is nonregular.

Closure of Regular Languages

We have certain operations such that, when applied to the languages, the resulting related language is also regular.

Definition (Homomorphism). Suppose two alphabets Σ and Γ . Consider a mapping $h : \Sigma \rightarrow \Gamma^*$. We further define $h : \Sigma^* \rightarrow \Gamma^*$ by principle of induction:

1. *Base case:* $h(\epsilon) = \epsilon$
2. *Inductive Step:* $h(a\omega) = h(a) \cdot h(\omega), \forall a \in \Sigma, \omega \in \Sigma^*$

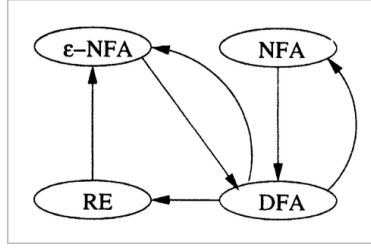
h is called a *homomorphism*. Intuitively, a homomorphism maps each symbol to another symbol in every string of the language.

Given regular languages L, L_1, L_2 :

- $L_1 \cup L_2$ is regular.
- $L_1 \cdot L_2$ is regular.
- $\bar{L} = \Sigma^* - L$ is regular.
- $L_1 \cap L_2$ is regular.
- $L_1 - L_2$ is regular.
- L^R is regular.
- $h(L)$ is regular, for some homomorphism h .

We may prove the results by constructing a regular expression through principle of induction on its length, from the base cases of a regular expression $\emptyset, \epsilon, a \in \Sigma$ and the defined operations.

Remark. Supposing L is a regular language over an alphabet Σ_1 . Then L is also a regular language over an alphabet $\Sigma_2 \supseteq \Sigma_1$, by considering the same DFA over Σ_2 .



Decision Problems of Regular Languages

We may examine some properties of regular languages through conversion between the equivalent forms of representing regular languages.

We check the **emptiness of regular languages** by considering the automaton, and checking whether any accepting states are reachable from the starting state. That is,

$$L = \emptyset \iff \forall q \in F, q \text{ is unreachable.}$$

We compute this using a recursive algorithm:

- *Base case:* The start state is reachable from the start state.
- *Inductive step:* If p is reachable from the start state, all states q such that $\exists a \in \Sigma, \delta(p, a) = q$ are reachable from the start state.

We check for **membership** by converting whichever representation available into a DFA, and then simulating the DFA. Alternatively, if given another automaton, we may attempt to simulate the NFA or ϵ -NFA itself, keeping track of all possible states.

Two automata A, A' are equivalent $\iff$ the minimisation of A is the same as the minimisation of A' , except for renaming of the states. (Minimisation of DFA, Subsection 2.4)

2.8 Linear Grammars

Definition (Right Linear Grammar). A **Right Linear Grammar** is a CFG where all productions are of the form:

$$\begin{aligned} A &\rightarrow \omega\beta & \omega \in T^*, \beta \in V \\ A &\rightarrow \omega & \omega \in T^* \end{aligned}$$

Similarly, a **Left Linear Grammar** is one in which all productions are of the form:

$$\begin{aligned} A &\rightarrow \beta\omega & \omega \in T^*, \beta \in V \\ A &\rightarrow \omega & \omega \in T^* \end{aligned}$$

Theorem. *Every regular language can be generated by a right-linear grammar.*

Proof. Suppose a regular language L . There exists a DFA $A = (Q, \Sigma, \delta, q_0, F)$ such that $L(A) = L$. WLOG, assume that $Q \cap \Sigma = \emptyset$. Consider the right-linear grammar $G = (Q, \Sigma, P, q_0)$, where:

$$1. \forall q, p \in Q, a \in \Sigma$$

$$\delta(q, a) = p \iff \exists \text{ a production } (q \rightarrow ap) \in P.$$

$$2. \forall q \in F$$

$$\exists \text{ a production } (q \rightarrow \epsilon) \in P.$$

We induct on the length $|\omega|$ to show that $\hat{\delta}(q_0, \omega) = p \iff q_0 \Rightarrow_G^* \omega p$.

1. *Basis Step:* For $|\omega| = 0$, $\omega = \epsilon$ and

$$\hat{\delta}(q_0, \omega) = p \iff p = q_0 \iff q_0 \Rightarrow_G^* \omega p$$

2. *Inductive Step:* Consider $\omega = ua$, for some $a \in \Sigma$ and $u \in \Sigma_{\leq k}$. We have:

$$\begin{aligned} \hat{\delta}(q_0, ua) = p' &\iff \exists p \in Q, \quad \hat{\delta}(q_0, u) = p \wedge \delta(p, a) = p' \\ &\iff \exists p \in Q, \quad q_0 \Rightarrow_G^* up \wedge p \rightarrow ap' \\ &\iff q_0 \Rightarrow_G^* uap' = \omega p' \end{aligned}$$

Theorem. A language generated by a right-linear grammar is regular. ■

Proof. Suppose a right-linear grammar $G = (V, \Sigma, P, S)$. WLOG, suppose $V \cap \Sigma = \emptyset$ and each productions are of the form $A \rightarrow bC$ or $A \rightarrow \epsilon$ where $A, C \in V$ and $b \in \Sigma \cup \{\epsilon\}$. Consider the NFA $A = (V, \Sigma, \delta, S, F)$, where:

$$(A \rightarrow aB) \in P \iff B \in \delta(A, a).$$

and $F = \{A \in V \mid (A \rightarrow \epsilon) \in P\}$. We induct on the length $|\omega|$ to show that $A \Rightarrow_G^* \omega B \iff B \in \hat{\delta}(A, \omega)$.

1. *Basis Step:* For $|\omega| = 0$, $\omega = \epsilon$ and

$$A \Rightarrow_G^* B \iff A = B \iff B \in \hat{\delta}(A, \epsilon).$$

2. Consider $\omega = ua$. Then

$$\begin{aligned} A \Rightarrow_G^* \omega B = uaB &\iff \exists C \in V, \quad A \Rightarrow_G^* uC \wedge C \Rightarrow_G aB \\ &\iff \exists C \in V, \quad \hat{\delta}(A, u) = C \wedge \delta(C, a) = B \\ &\iff \hat{\delta}(A, ua) = \hat{\delta}(A, \omega) = B \end{aligned}$$

■

3 Context Free Languages

We now consider a larger class of languages known as the **context-free languages**. These languages can be defined by a recursive notation known as *context-free grammars*, or a automaton-like notation known as *pushdown automaton*.

Intuitively, the class of CFLs comprises the RLs, and languages in which symmetry is required, and certain forms of matching (in opposite directions).

3.1 Context-Free Grammars

Definition (CFG). A CFG $C = (V, T, P, S)$, where:

- V : A finite set of non-terminals
- T : A finite set of terminals
- P : A finite set of productions
- S : A start symbol, $S \in V$

A production is of the form $A \rightarrow \gamma$, where $A \in V$ and $\gamma \in (V \cup T)^*$. If we have the productions of $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_n$, we may write $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$.

Example ($L(0^*1(0+1)^*)$). To generate the regular language $L(0^*1(0+1)^*)$, consider the CFG $G = (\{S, A, B\}, \{0, 1\}, P, S)$, where P comprises the productions:

$$S \rightarrow AB, \quad A \rightarrow 0A | \epsilon, \quad B \rightarrow 0B | 1B | \epsilon.$$

Derivations

To infer if a certain string is in the language of the CFG, we may apply *recursive inference* or *derivation*.

Definition (Recursive Inference). We may construct the string recursively from body to head. For example, given a production $A \rightarrow BC$, we take strings $b \in L(B)$ and $c \in L(C)$ and concatenate them in the proper order and infer that the resulting string is in $L(A)$.

	String Inferred	In language of	Production used	String(s) used
(i)	ϵ	A	$A \rightarrow \epsilon$	–
(ii)	ϵ	B	$B \rightarrow \epsilon$	–
(iii)	0	A	$A \rightarrow 0A$	(i)
(iv)	0	B	$B \rightarrow 0B$	(ii)
(v)	1	B	$B \rightarrow 1B$	(ii)
(vi)	00	A	$A \rightarrow 0A$	(iii)
(vii)	10	B	$B \rightarrow 1B$	(iv)
(viii)	11	B	$B \rightarrow 1B$	(v)
(ix)	11	S	$S \rightarrow AB$	(i), (viii)
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

This procedure is known as **recursive inference**.

Notation ($\Rightarrow$). If $A \in V$ such that $A \rightarrow \gamma$ is a production of CFG G , we denote

$$\alpha A \beta \Rightarrow_G \alpha \gamma \beta \quad \forall \alpha, \beta \in (V \cup T)^*.$$

Definition (Derivation). We can expand S with one of its productions, and recursively expand each non-terminal with the body of its productions until a string of terminals is derived.

$$S \Rightarrow AB \Rightarrow A1B \Rightarrow A11B \Rightarrow A110B \Rightarrow A110 \Rightarrow 0A110 \Rightarrow 0110$$

This is referred to as a **derivation** of 0110. We may then define a derivation relation $\Rightarrow_G^*$:

1. *Base Case*: $\alpha \Rightarrow_G^* \alpha$
2. *Inductive Step*: If $\alpha \Rightarrow_G^* \beta \wedge \beta \Rightarrow_G \gamma$, then $\alpha \Rightarrow_G^* \gamma$.

That is, there exists a derivation from α to β .

Definition (Left/Right-most Derivations). A **leftmost derivation** is one in which at each step, the leftmost variable is replaced by one of its production bodies. This restricts the number of choices we have in deriving a string. Similarly, we can also have a **rightmost derivation**.

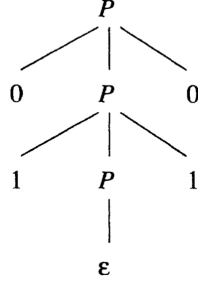
Definition (Sentential Forms). Any string $\alpha \in (V \cup T)^*$ such that $S \Rightarrow^* \alpha$ is called a **sentential form**, a string (of terminals or non-terminals) derived from the start symbol. We further have the analogue for left/right-most derivations, known as a *left/right-sentential form*.

Parse Trees

Definition (Parse Tree). We use a tree representation for derivations, where each interior node is labelled by a nonterminal in V , each leaf is labelled by a symbol in $V \cup T$, or ϵ , and the children $X_1, X_2, \dots, X_k$ (from the left) of an interior node A forms the body of the production $(A \rightarrow X_1 X_2 \dots X_k) \in P$. This is known as the **parse tree**. We call the in-order traversal of its leaves the **yield** of the parse tree.

Consider the parse trees whose yields are terminal strings, and the root is labelled by the start symbol. Then the yield is a string in the language of the grammar.

Remark. Derivations are non unique, but parse trees and its equivalent left-most derivations, right-most derivations are unique for unambiguous grammars.



Language of a CFG

Definition (Language of a CFG). The **language** of a CFG G is denoted $L(G)$, and is defined by

$$L(G) = \{\omega \in T^* \mid S \Rightarrow_G^* \omega\},$$

the set of strings that can be derived from the start symbol in finitely many steps.

Remark. The language of a CFG is hence the set of all sentential forms that consist solely of terminals.

Definition (Context Free Language). If L is $L(G)$ for some CFG G , then L is a **context-free language**.

We prove a property of strings in a specified CFL by inducting on the length of the string to prove membership, and inducting on the number of steps of derivation to prove the specified property, as illustrated by the following example.

Example (Property of Context-Free Language). Suppose the CFG $G = (\{S\}, \{0, 1\}, P, S)$, where P consists the productions

$$P \rightarrow \epsilon \mid 0 \mid 1 \mid 0P0 \mid 1P1.$$

Then $L(G)$ is the set of palindromes over $\{0, 1\}$.

Proof. ($\Rightarrow$) Suppose ω is a palindrome. We prove by induction on $|\omega|$ that ω is in $L(G)$.

1. *Basis Step:* If $|\omega| = 0$ or $|\omega| = 1$, then $\omega = \epsilon, 0, 1$. Since we have the productions $P \rightarrow \epsilon \mid 0 \mid 1$, $P \Rightarrow^* \omega$.
2. *Inductive Step:* Suppose $|\omega| \geq 2$. Since ω is a palindrome, it begins and end with the same symbol. That is, $\omega = 0x0$ or $\omega = 1x1$, where x is a palindrome over $\{0, 1\}$. By inductive hypothesis, $P \Rightarrow^* x$.
 - If $\omega = 0x0$, We have $P \Rightarrow 0P0 \Rightarrow^* 0x0 = \omega$.
 - If $\omega = 1x1$, We have $P \Rightarrow 1P1 \Rightarrow^* 1x1 = \omega$. A

We have $P \Rightarrow^* \omega$ is in $L(G)$.

($\Leftarrow$) Suppose $\omega \in L(G)$. We prove by induction on the number of steps in a derivation of ω that it is a palindrome.

1. *Basis Step:* For one-step derivations, we have $\epsilon, 0, 1$ which are all palindromes.
2. *Inductive Step:* Suppose that strings x generated by n -step derivations are palindromes where $n \geq 1$. Consider an $(n+1)$ -step derivation of ω . Then $\omega = 0x0$ or $\omega = 1x1$ since $n+1 \geq 2$, and these are the only productions allowing additional steps of a derivation. Since x is a palindrome, ω is also a palindrome. ■

Inference, Derivations, and Parse Trees

Given a grammar $G = (V, T, P, S)$, the following are equivalent:

- The recursive inference procedure determines $\omega \in T^*$ is in the language of $A \in V$
- $A \Rightarrow_G^* \omega$
- There exists a left/right-most derivation from A to ω
- There exists a parse tree with root A and yield ω

3.2 Ambiguous Grammars

Definition (Ambiguous Grammar). A grammar G is said to be ambiguous if, there exists at least two different parse trees with root labelled S and yield $\omega \in T^*$. Equivalently, G is ambiguous if there exist at least two left/right-most derivations.

Remark. The ambiguity of a grammar is undecidable. There is no algorithm to decide the ambiguity of a CFG, and no general algorithm to remove ambiguity. However, there exists certain techniques to eliminate certain kinds of ambiguity.

Eliminating Ambiguity

Example. Suppose the CFG $G = (\{E, I, S\}, \{a, b, 0, 1\}, P, S)$ where P consists the productions

$$E \rightarrow I|E + E|E * E|(E) \quad I \rightarrow a|b|Ia|Ib|I0|I1.$$

This grammar is ambiguous for two causes: (1) The precedence of operators is not defined, and (2) a sequence of identical operators can be grouped from the left or the right. To eliminate ambiguity, we impose an order of precedence, and left-associativity of operators.

Precedence can be enforced by introducing different variables, each of which represents expressions sharing the same level of “binding strength”:

- Identifiers and paranthesised expressions are *factors*, expressions that cannot be broken apart by any adjacent operator.
- A product of one or more factors are *terms*, expressions that cannot be broken by the $+$ operator.
- A sum of one or more terms is then all other possible *expressions* to be represented.

We hence disambiguate with the grammar

$$\begin{aligned} I &\rightarrow a|b|Ia|Ib|I0|I1 \\ F &\rightarrow I|(E) \\ T &\rightarrow F|T * F \\ E &\rightarrow T|E + T \end{aligned}$$

Remark. It is difficult to prove disambiguity of a specified grammar.

Inherent Ambiguity

Definition (Inherently Ambiguous). A CFL L is **inherently ambiguous** if all its grammars are ambiguous.

Example (Inherently Ambiguous CFL).

$$L = \{a^n b^n c^m d^m | n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n | n \geq 1, m \geq 1\}$$

is an inherently ambiguous language.

Remark. Proof of inherent ambiguity is complex.

3.3 Simplification of CFG

Prerequisites

For a grammar $G = (V, T, P, S)$, we have the following definitions:

Definition (Generating Symbols). A symbol A is a *generating symbol* if $\exists \omega \in T^*, A \Rightarrow^* \omega$.

Definition (Reachable Symbols). A symbol A is a *reachable symbol* if $\exists \alpha, \beta \in (V \cup T)^*, S \Rightarrow^* \alpha A \beta$.

Definition (Useless Symbols). A symbol $a \in V \cup T$ is a *useless symbol* if it does not appear in any derivation of a terminal string from the start symbol $S \Rightarrow^* \omega$ for any $\omega \in T^*$.

A symbol A is useful (not useless) $\implies$ A is reachable and generating. The converse is not always true.

Definition (ϵ -Productions). An ϵ -production is a production of the form $A \rightarrow \epsilon$ for some $A \in V$.

Definition (Nullable Symbols). A symbol A is a *nullable symbol* if $A \Rightarrow^* \epsilon$.

Definition (Unit Productions). A unit production is a production of the form $A \rightarrow B$ for some $A, B \in V$.

Definition (Unit Pairs). A tuple (A, B) for some $A, B \in V$ is a unit pair if $A \Rightarrow^* B$.

CFG Simplification Algorithm

Suppose we are given a grammar $G = (V, T, P, S)$ such that $L(G) \neq \emptyset$. Let $G_3 = (V_3, T_3, P_3, S)$ be the grammar obtained by **eliminating all ϵ -productions** by first finding all nullable symbols by iterating the following inductive algorithm:

1. *Basis Step*: If $A \rightarrow \epsilon$ is a production of G , A is nullable.
2. *Inductive Step*: If $B \rightarrow \beta$ is a production and β consists only of nullable nonterminals, B is nullable.

For each production $A \rightarrow X_1 X_2 \cdots X_k$, given m of the k X_i s are nullable, create 2^m versions of the production of all possible combinations of presence/absence of the nullable X_i s. If all symbols are nullable, the case of $A \rightarrow \epsilon$ is not included.

Example (Removing ϵ -productions). Suppose $S \rightarrow AB$, where A, B are nullable symbols. In the new grammar, we have the productions $S \rightarrow AB|A|B$.

Then G_3 does not contain any ϵ -productions, and $L(G_3) = L(G) - \{\epsilon\}$.

Proof. We prove the more general statement that

$$A \Rightarrow_{G_3}^* \omega \iff A \Rightarrow_G^* \omega \wedge \omega \neq \epsilon.$$

($\implies$) Suppose $A \Rightarrow_{G_3}^* \omega \neq \epsilon$ since G_3 has no ϵ -productions. Inducting on the number of steps of the derivation $A \Rightarrow_{G_3}^* \omega$:

1. *Basis Step*: There is a production $A \rightarrow \omega$ in G_3 . By construction of G_3 , there is some production $A \rightarrow \alpha$ in G such that α is ω with some number of nullable symbols interspersed. We have $A \Rightarrow_G \alpha \Rightarrow_G^* \omega$ by deriving ϵ from all the nullable symbols.
2. *Inductive Step*: Suppose a derivation of $n > 1$ steps. The derivation must look like

$$A \Rightarrow_{G_3} X_1 X_2 \cdots X_k \Rightarrow_{G_3}^* \omega.$$

Similarly, this must come from a production $A \rightarrow Y_1 Y_2 \cdots Y_m$ in G , where the Y s are the X s in order with some nullable symbols interspersed. Let $\omega = \omega_1 \omega_2 \cdots \omega_k$, where $X_i \Rightarrow_{G_3}^* \omega_i$. Each derivation takes fewer than n steps, and hence $X_i \Rightarrow_G^* \omega_i$ by inductive hypothesis.

$$A \Rightarrow_G Y_1 Y_2 \cdots Y_m \Rightarrow_G^* X_1 X_2 \cdots X_k \Rightarrow_G^* \omega_1 \omega_2 \cdots \omega_k = \omega.$$

($\impliedby$) Suppose $A \Rightarrow_G^* \omega$ and $\omega \neq \epsilon$. Inducting on the number of steps of the derivation $A \Rightarrow_G^* \omega$:

1. *Basis Step*: There is a production $A \rightarrow \omega$ in G . Since $\omega \neq \epsilon$, it is also a production of G_3 , and $A \Rightarrow_{G_3}^* \omega$.

2. *Inductive Step*: Suppose a derivation of $n > 1$ steps. The derivation must look like $A \Rightarrow_G Y_1 Y_2 \cdots Y_m \Rightarrow_G^* \omega$. We can break up $\omega = \omega_1 \omega_2 \cdots \omega_m$ such that $Y_i \Rightarrow_G^* \omega_i$. Let $X_1, X_2, \dots, X_k$ be the Y_j s such that $\omega_j \neq \epsilon$. Since $\omega \neq \epsilon$, $k \geq 1$ and $A \rightarrow X_1 X_2 \cdots X_k$ is a production in G_3 . Each derivation $Y_j \Rightarrow_G^* \omega_j$ takes fewer than n steps, and hence if $\omega_j \neq \epsilon$, $Y_j \Rightarrow_{G_3}^* \omega_j$.

$$A \Rightarrow_{G_3} X_1 X_2 \cdots X_k \Rightarrow_{G_3}^* \omega.$$

We have $S \Rightarrow_{G_3}^* \omega \iff A \Rightarrow_G^* \omega \wedge \omega \neq \epsilon$. Hence $L(G_3) = L(G) - \{\epsilon\}$. ■

Let $G_4 = (V, T, P_4, S)$ be the grammar obtained by **eliminating all unit productions** by first finding all unit pairs by iterating the following inductive algorithm:

1. *Basis Step*: $\forall A \in V$, (A, A) is a unit pair.
2. *Inductive Step*: If (A, B) is a unit pair and $B \rightarrow C$ is a production, for some nonterminal C , (A, C) is a unit pair.

For each unit pair (A, B) , let $\{A \rightarrow \alpha \mid B \rightarrow \alpha \text{ is a non unit production in } P\} \subseteq P'$. Then G' does not contain any unit productions, and $L(G_4) = L(G)$.

Let $G_1 = (V_1, T_1, P_1, S)$ be the grammar obtained with the following steps:

1. **Eliminating non-generating symbols** and all productions involving one or more of those symbols. We find all generating symbols by iterating the following inductive algorithm:
 - (a) *Basis Step* All terminals $a \in T$ are generating.
 - (b) *Inductive Step*: If $A \rightarrow \alpha$ is a production and α consists only of generating symbols, A is generating.
2. **Eliminating non-reachable symbols** in the resultant grammar G_2 . We find all reachable symbols by iterating the following inductive algorithm:
 - (a) *Basis Step* S is reachable.
 - (b) *Inductive Step*: If A is reachable and $A \rightarrow \alpha$ is a production, every symbol in α is reachable.

Then G_1 has no useless symbols and $L(G_1) = L(G)$.

Proof. Suppose a symbol in G_1 , $X \in V_1 \cup T_1$. X is generating, and every symbol used in the derivation $X \Rightarrow^* \omega$ is also generating. We have $X \Rightarrow_{G_2}^* \omega$.

Since X is not eliminated in the second step, $\exists \alpha, \beta \in (V \cup T)^*$, $S \Rightarrow_{G_2}^* \alpha X \beta$, and every symbol in the derivation is also reachable, hence $S \Rightarrow_{G_1}^* \alpha X \beta$.

Since every symbol in $\alpha X \beta$ is reachable and generating (since they are in $V_2 \cup T_2$)

$$S \Rightarrow_{G_1}^* \alpha X \beta \Rightarrow_{G_1}^* xwy$$

for some $x, \omega, y \in T^*$. X is useful in G_1 .

Additionally, $L(G_1) \subseteq L(G)$ since we have only eliminated symbols and productions. $L(G) \subseteq L(G_1)$ since every derivation $S \Rightarrow_G^* \omega$ consists of only useful symbols, hence $S \Rightarrow_{G_1}^* \omega$. $L(G) = L(G_1)$. ■

Supposing there is at least one string in $L(G)$, by following this procedure of (in this order):

1. Eliminating ϵ -productions
2. Eliminating unit productions

3. Eliminating useless symbols

we obtain a simplified CFG G' with the various simplifications, with $L(G') = L(G) - \{\epsilon\}$.

Remark. To obtain an equivalent CFG if $\epsilon \in L(G)$, we specify the CFG $G'' = (V \cup \{S_0\}, T, P'', S_0)$, where

$$P'' = P \cup \{S_0 \rightarrow S, S_0 \rightarrow \epsilon\}.$$

Normal Forms

Definition (Chomsky Normal Form). A CFG $G = (V, T, P, S)$ is said to be in *Chomsky Normal Form* if all productions are of the form

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a$$

where $A, B, C \in V$ and $a \in T$, and G has no useless symbols.

CNF guarantees non-decreasing length of the derived string, with respect to the number of productions used.

Theorem. *Every nonempty language without ϵ is $L(G)$ for some CFG G in Chomsky Normal Form.*

Given the simplified CFG without ϵ -productions, unit productions and useless symbols, we can obtain an equivalent CFG in CNF as follows: Suppose a production

$$A \rightarrow X_1 X_2 \cdots X_k.$$

We break the production into the following set of productions:

$$\begin{aligned} A &\rightarrow Z_1 B_2 \\ B_2 &\rightarrow Z_2 B_3 \\ &\vdots \\ B_{k-1} &\rightarrow Z_{k-1} Z_k \\ Z_i &\rightarrow X_i, \quad \text{if } X_i \in T \\ Z_i &= X_i, \quad \text{if } X_i \in V \end{aligned}$$

with new nonterminals B_i and (possibly) Z_i .

Theorem (Size of Parse Tree). *Suppose a parse tree of a Chomsky-Normal-Form Grammar. If the length of the longest path from the root to a node is k , the size of the string generated is at most 2^{k-1} .*

Definition (Greibach Normal Form). A CFG $G = (V, T, P, S)$ is said to be in *Greibach Normal Form* if all productions are of the form

$$A \rightarrow a\alpha.$$

where $A \in V$, $a \in T$ and $\alpha \in (V \cup T)^*$.

GNF guarantees a generated terminal in every production used as the leftmost symbol.

Theorem. *Every nonempty language without ϵ is $L(G)$ for some CFG G in Greibach Normal Form.*

3.4 Pushdown Automata

A pushdown automaton is in essence a NFA with ϵ -transitions and a stack on which it can store a string of “stack symbols”.

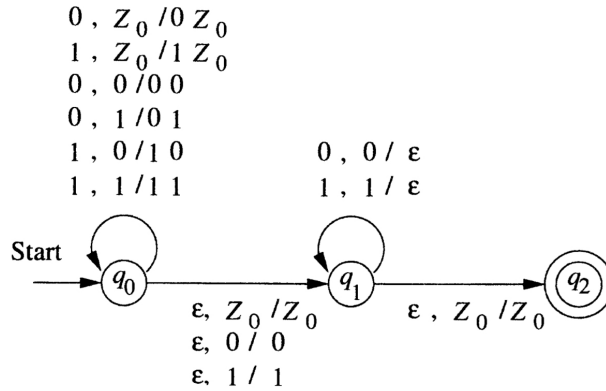
Definition (PDA). A PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where:

- Q : A finite set of states

- Σ : A finite set of input symbols
- Γ : A finite stack alphabet
- δ : A transition function taking as argument a triple (p, a, X) representing the current state of the PDA and returns a set of pairs (q, γ) of new states and replacement stack symbols. $\delta : Q \times \Sigma \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$
- q_0 : A start state, $q_0 \in Q$
- Z_0 : A initial symbol on the stack, $Z_0 \in \Gamma$
- F : A set of final or accepting states. $F \in \mathcal{P}(Q)$

We have a transition diagram for PDAs, in which:

- Nodes correspond to states of the PDA
- Doubly circled states are accepting
- An arrow labelled “Start” indicates the start state
- Each arc corresponds to a transition of the PDA:
An arc labelled $a, X/\alpha$ from state p to q indicates $(p, \alpha) \in \delta(q, a, X)$
- By convention, we take Z_0 to be the start symbol



Definition (Instantaneous Descriptions). The **ID** of a PDA is the triple (q, ω, γ) such that q is the current state, ω the remaining input and γ the stack contents.

Notation ($\vdash$). If $(p, \alpha) \in \delta(q, a, X)$, we denote

$$(q, a\omega, X, \beta) \vdash (p, \omega, \alpha\beta) \quad \forall \omega \in \Sigma^*, \beta \in \Gamma^*.$$

This represents one move of the PDA.

Notation ($\vdash_P^*$). To represent zero or moves of the PDA, we further define the relation $\vdash_P^*$:

1. *Base Case*: $I \vdash_P^* I$
2. *Inductive Step*: For IDs I, J , if ϵ some ID K such that $I \vdash K$ and $K \vdash_P^* J$, then $I \vdash_P^* J$

Language of a PDA

Definition (Language of a PDA). The *language* of a PDA P is denoted $L(P)$ and depends on the modality of acceptance of the PDA. It is defined by

$$\{\omega \mid (q_0, \omega, Z_0) \vdash_P^* (q_f, \epsilon, \alpha)\}, q_f \in F \quad \text{for acceptance by final state.}$$

$$\{\omega \mid (q_0, \omega, Z_0) \vdash_P^* (q_f, \epsilon, \epsilon)\}, q_f \in F \quad \text{for acceptance by empty stack.}$$

Theorem (Equivalence of Acceptance Modalities). *The two modalities of acceptance are equivalent, in the sense that L has a PDA that accepts it by final state $\iff L$ has a PDA that accepts it by empty stack. Equivalently, the class of languages accepted by final state is equal to the class of languages accepted by empty stack.*

Theorem. *Suppose a language L has a PDA that accepts it by empty stack. Then L has a PDA that accepts it by final state.*

Proof. Suppose $L = L(P_N)$ through acceptance by empty stack for some PDA $P_N = (Q, \Sigma, \Gamma, \delta_N, q_0, Z_0, F)$. Consider the PDA

$$P_F = (Q \cup \{p_0, p_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta_F, p_0, X_0, \{p_f\})$$

such that

1. P_F spontaneously transits to the start state of P_N , pushing the start symbol.

$$\delta_F(p_0, \epsilon, X_0) = \{(q_0, Z_0 X_0)\}$$

2. P_F simulates P_N .

$$\forall q \in Q, a \in \Sigma \cup \{\epsilon\}, Y \in \Gamma \quad \delta_N(q, a, Y) \subseteq \delta_F(q, a, Y)$$

3. After the stack is empty, transit to the final state of P_F .

$$\forall q \in Q \quad (p_f, \epsilon) \in \delta_F(q, \epsilon, X_0)$$

We now show that $w \in L(P_F)$ with acceptance by final state $\iff w \in L$.

($\Leftarrow$) Suppose $w \in L$. $(q_0, w, Z_0) \vdash_{P_N}^*$ for some state q .

$$(q_0, w, Z_0 X_0) \vdash_{P_N}^* (q, \epsilon, X_0).$$

Since P_F has all the moves of P_N , $(q_0, w, Z_0 X_0) \vdash_{P_F}^* (q, \epsilon, X_0)$. Hence we have

$$(p_0, w, X_0) \vdash_{P_F} (q_0, w, Z_0 X_0) \vdash_{P_F}^* (q, \epsilon, X_0) \vdash_{P_F} (p_f, \epsilon, \epsilon)$$

and P_F accepts w by final state.

($\Rightarrow$) Suppose P_F accepts w by final state. Since transitions to p_f require X_0 as the stack symbol, and X_0 can only be on the bottommost position, any computation of P_F that accepts w must be of the previously shown sequence. Hence w is in P_N . ■

Theorem. *Suppose a language L has a PDA that accepts it by final state. Then L has a PDA that accepts it by empty stack.*

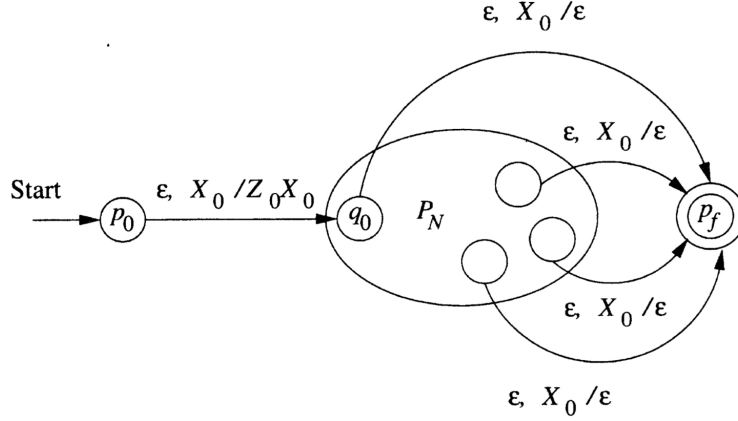
Suppose $L = L(P_F)$ through acceptance by final state for some PDA $P_F = (Q, \Sigma, \Gamma, \delta_F, q_0, Z_0, F)$. Consider the PDA

$$P_N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta_N, p_0, X_0)$$

such that

1. We push the start symbol of P_F onto the stack and going to the start state of P_F .

$$\delta_N(p_0, \epsilon, X_0) = \{q_0, Z_0 X_0\}$$



2. P_N simulates P_F .

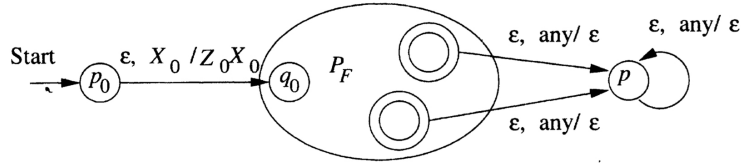
$$\forall q \in Q, a \in \Sigma \cup \{\epsilon\}, Y \in \Gamma \quad \delta_F(q, a, Y) \subseteq \delta_N(q, a, Y)$$

3. If P_F accepts, P_N starts emptying its stack without consuming input.

$$\forall q \in F, Y \in \Gamma \cup \{X_0\} \quad (p, \epsilon) \in \delta_N(q, \epsilon, Y)$$

4. After P_F has accepted, P_N pops every symbol until the stack is empty.

$$\forall Y \in \Gamma \cup \{X_0\} \quad \delta_N(p, \epsilon, Y) = \{(p, \epsilon)\}$$



It can be shown that $\omega \in L(P_N)$ with acceptance by empty stack $\iff \omega \in L$.

Remark. For a given PDA P , the languages that P accepts by final state and by empty stack are different in general.

3.5 CFGs vs PDAs

Equivalence of CFGs and PDAs

Theorem. A language L is accepted by some PDA if L is accepted by some CFG.

Proof. Suppose a CFG $G = (V, T, G, S)$. We construct a PDA P as follows:

$$P = (\{q\}, T, V \cup T, \delta, q, S)$$

where δ is defined:

1. For each nonterminal $A \in V$, $\delta(q, \epsilon, A) = \{(q, \beta) \mid A \rightarrow \beta \in G\}$
2. For each terminal $a \in T$, $\delta(q, a, a) = \{q, \epsilon\}$

We prove that $\omega \in L(G) \iff \omega \in L(P)$ with acceptance by empty stack.

($\implies$) Suppose $\omega \in L(G)$. Then ω has a leftmost derivation

$$S = \gamma_1 \Rightarrow \gamma_2 \Rightarrow \cdots \Rightarrow \gamma_n = \omega.$$

We show by induction on i that $(q, \omega, S) \vdash^* (q, y_i, \alpha_i)$, where $\gamma_i = x_i \alpha_i$, $\omega = x_i y_i$, $x_i, y_i \in T^*$ and $\alpha_i \in (V \cup T)^*$. That is, α_i is the tail of γ_i from the first nonterminal (from the left), and y_i is the nonterminal string to be generated from α_i .

1. *Basis Step:* $\gamma_1 = S$, $x_1 = \epsilon$, $y_1 = \omega$. By definition, $(q, \omega, S) \vdash^* (q, \omega, S)$.
2. *Inductive Step:* Suppose by inductive hypothesis that $(q, \omega, S) \vdash^* (q, y_i, \alpha_i)$. α_i begins with a nonterminal A . By replacing A by one of its production bodies, say β with rule (1) of δ , and removing all terminals at the top of the stack with rule (2) of δ , we reach the ID $(q, y_{i+1}, \alpha_{i+1})$.

Since ω is a string of terminals, $\alpha_n = \epsilon$, and we have $(q, \omega, S) \vdash^* (q, \epsilon, \epsilon)$. P accepts ω by empty stack.

($\impliedby$) Suppose $(q, x, A) \vdash^* (q, \epsilon, \epsilon)$. Inducting on the number of moves taken by P , we have:

1. *Basis Step:* $A \rightarrow \epsilon$ is a production of G , and the move is of rule (1). We have $x = \epsilon$ and $A \Rightarrow \epsilon$.
2. *Inductive Step:* Suppose P takes n moves. The first move must be of rule (1), where A is replaced by one of its production bodies. Let the production used be $A \rightarrow Y_1 Y_2 \cdots Y_k$ for some $Y_i \in (V \cup T)^*$. By taking $x = x_1 x_2 \cdots x_k$, where each x_i is the portion of the input consumed until Y_1 is popped off the stack, for all $i = 1, 2, \dots, k$:

$$(q, x_i x_{i+1} \cdots x_k, Y_i) \vdash^* (q, x_{i+1} \cdots x_k, \epsilon).$$

Each of these sequences are no more than $n - 1$ moves (since the PDA completes in $n - 1$ moves), $Y_i \Rightarrow^* x_i$ for nonterminals Y_i . For terminals Y_i , only one move is involved, matching the one symbol against Y_i . Hence we have the derivation:

$$A \Rightarrow Y_1 Y_2 \cdots Y_k \Rightarrow^* x_1 Y_2 \cdots Y_k \Rightarrow^* \cdots \Rightarrow^* x_1 x_2 \cdots x_k.$$

We have $A \Rightarrow^* x$.

By letting $A = S$ and $x = \omega$, since $\omega \in L(P)$ by acceptance by empty stack, $S \Rightarrow \omega$ and it is in $L(G)$. ■

Theorem. A language L is accepted by some CFG if L is accepted by some PDA.

Proof. Suppose a PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ accepting by empty stack. We construct a CFG as follows:

$$G = (V, \Sigma, R, S)$$

where $V = \{S\} \cup \{[pXq] \mid p, q \in Q, X \in \Gamma\}$. Each nonterminal encodes the ‘net popping’ of symbol X from the stack, with a change in state from p to q . The productions of G are then:

1. $\forall p \in Q$, $S \rightarrow [q_0 Z_0 p] \in R$. That is, we generate all strings that pop Z_0 while going from state q_0 to (any) state p , $(q_0, \omega, Z_0) \vdash^* (p, \epsilon, \epsilon)$.
2. If $(r, Y_1 Y_2 \cdots Y_k) \in \delta(q, a, X)$, where $a \in \Sigma \cup \{\epsilon\}$, $k \in \mathbb{N}$,

$$\forall (r_1, r_2, \dots, r_k) \in Q^k, \quad [qXr_k] \rightarrow a[rY_1 r_1][r_1 Y_2 r_2] \cdots [r_{k-1} Y_k r_k].$$

Intuitively, one of the ways to pop X and go from states q to r_k is to read a and then use some input to sequentially pop each of the produced symbols.

We prove that $(q, \omega, X) \vdash^* (p, \epsilon, \epsilon) \iff [qXp] \Rightarrow^* \omega$.

($\implies$) Suppose $(q, \omega, X) \vdash^* (p, \epsilon, \epsilon)$. Inducting on the number of moves taken by P , we have:

1. *Basis Step:* With one step, $(p, \epsilon) \in \delta(q, \omega, X)$ and ω is a single symbol or ϵ . By construction, $[qXp] \rightarrow \omega$ is a production and $[qXp] \Rightarrow \omega$.
2. *Inductive Step:* Suppose the sequence $(q, \omega, X) \vdash^* (p, \epsilon, \epsilon)$ takes $n > 1$ steps. The sequence must be of the form

$$(q, \omega, X) \vdash (r_0, x, Y_1 Y_2 \cdots Y_k) \vdash^* (p, \epsilon, \epsilon)$$

where $\omega = ax$, $a \in \{\epsilon\} \cup \Sigma$. We have $(r_0, Y_1 Y_2 \cdots Y_k) \in \delta(q, a, X)$ and by construction, we have the productions $[qXp] \rightarrow a[r_0 Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{k-1} Y_k p]$ for all states $r_1, r_2, \dots, r_k \in Q$. Let $x = \omega_1 \omega_2 \cdots \omega_k$ where each ω_i is the input consumed to ‘pop’ Y_i off the stack. We have the sequences of less than n moves $(r_{i-1}, \omega_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon)$ and hence $[r_{i-1} Y_i r_i] \Rightarrow^* \omega_i$ by inductive hypothesis.

$$[qXr_k] \Rightarrow a[r_0 Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{k-1} Y_k p] \Rightarrow^* a\omega_1[r_1 Y_2 r_2][r_2 Y_3 r_3] \cdots [r_{k-1} Y_k p] \Rightarrow^* a\omega_1 \omega_2 \cdots \omega_k.$$

We have $[qXp] \Rightarrow^* \omega$.

($\impliedby$) Suppose $[qXp] \Rightarrow^* \omega$. Inducting on the number of steps in the derivation, we have:

1. *Basis Step:* With one step, $[qXp] \rightarrow \omega$ is a production $\iff$ there is a transition of P in which X is popped and we transit states from q to p . $(p, \epsilon) \in \delta(q, a, X)$ and $a = \omega$, hence $(q, \omega, X) \vdash (p, \epsilon, \epsilon)$.
2. *Inductive Step:* Suppose $[qXp] \Rightarrow^* \omega$ by $n > 1$ steps. The derivation must be of the form

$$[qXp] \Rightarrow a[r_0 Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{k-1} Y_k p] \Rightarrow^* \omega.$$

We have $(r_0, Y_1 Y_2 \cdots Y_k) \in \delta(q, a, X)$. Let $\omega = a\omega_1 \omega_2 \cdots \omega_k$ such that $[r_{i-1} Y_i r_i] \Rightarrow^* \omega_i$. By inductive hypothesis, $(r_{i-1}, \omega_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon)$ for each i . This gives us

$$(q, a\omega_1 \omega_2 \cdots \omega_k, X) \vdash (r_0, \omega_1 \omega_2 \cdots \omega_k, Y_1 Y_2 \cdots Y_k) \vdash^* (r_1, \omega_2 \omega_3 \cdots \omega_k, Y_2 Y_3 \cdots Y_k) \vdash^* \cdots \vdash^* (p, \epsilon, \epsilon).$$

Since $S \Rightarrow^* \omega \iff [q_0 Z_0 p] \Rightarrow^* \omega$ for some state p (by construction), and each $[q_0 Z_0 p] \Rightarrow^* \omega \iff (q_0, \omega, Z_0) \vdash^* (p, \epsilon, \epsilon)$, we have $L(G) = L(P)$ with acceptance by empty stack.

3.6 Context Free Languages

To show that a given language is non context-free, we may use the pumping lemma. Intuitively, in any sufficiently long string, it is possible to find two short nearby substrings that we can “pump” in tandem, while keeping the resulting string in L .

Lemma. *Let L be a CFL. Then $\exists n = f(L)$ such that $\forall z \in L$, $|z| \geq n$, $\exists u, v, w, x, y \in \Sigma^*$, $z = uvwxy$ such that:*

1. $|vwx| \leq n$. That is, the two strings to be “pumped” is not too far from the other.
2. $vx \neq \epsilon$. At least one of the strings to be “pumped” cannot be empty.
3. $\forall i \geq 0$, $uv^iwx^iy \in L$

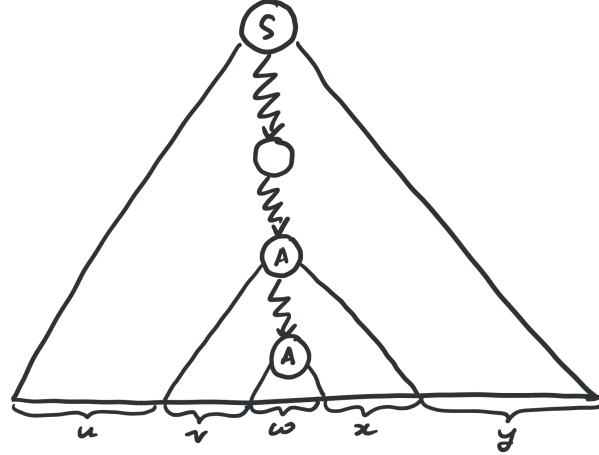
The contrapositive of the lemma is used to prove that a language is not context-free. That is, $\forall n \in \mathbb{N}$, $\exists z \in L$, such that $|z| \geq n$, $\forall u, v, w, x, y \in \Sigma^*$, where $z = uvwxy$, $|vwx| \leq n$, $vx \neq \epsilon$, we have

$$\exists i \in \mathbb{N} (uv^iwx^iy \notin L)$$

Proof. Suppose L is context-free. Without loss of generality, assume $L \neq \emptyset$, $L \neq \{\epsilon\}$ and a CNF grammar $G = (V, T, P, S)$ for $L - \{\epsilon\}$. Let $m = |V|$ and $n = 2^m$. Consider a string $z \in L$ such that $|z| \geq n$. Its parse tree must have a path from the root to a leaf of length at least $m + 1$. By Pigeonhole Principle, there must exist a repeated nonterminal A .

We have the derivation of the form, where $z = uvwxy$ for some $u, v, w, x, y \in \Sigma^*$,

$$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uvwxy.$$



Additionally, we have $A \Rightarrow^* vAx$ and $A \Rightarrow^* w$. For any $i \in \mathbb{N}$, we thus may have the derivation

$$S \Rightarrow^* uAy \Rightarrow^* uv^i Ax^i y \Rightarrow^* uv^i wx^i y.$$

With the assumption that the grammar is of CNF, since the repeated terminals are maximally m production steps apart, we have the length $|vwx| \leq n$, and since the derivation $A \Rightarrow^* vAx$ uses at least one production step, $vx \neq \epsilon$.

For the excluded cases when constructing the CNF grammar, the statement of the pumping lemma is vacuously true for $L = \emptyset$ and we choose n such that $z \neq \epsilon$ for $L = \{\epsilon\}$. ■

By using the pumping lemma, we can construct a proof of the following structure to show a specified language is non context-free:

Proposition. $L = \{a^m b^m c^m | m \geq 1\}$ is not context-free.

Proof. Suppose L is regular. Let n be as in the Pumping Lemma. Let $z = a^n b^n c^n$. Let $z = uvwxy$ as in the pumping lemma. Since $|vwx| \leq n$, vwx cannot contain both a and c .

Suppose vwx does not contain a 's. By the pumping lemma, $uv^2wx^2y \in L$. But uv^2wx^2y contains n a 's, and $|uv^2wx^2y| > 3n$, hence $uv^2wx^2y \notin L$.

Suppose vwx does not contain c 's. Similarly, By the pumping lemma, $uv^2wx^2y \in L$. But uv^2wx^2y contains n c 's, and $|uv^2wx^2y| > 3n$, hence $uv^2wx^2y \notin L$.

Hence L is non context-free. ■

Proposition. $L = \{\alpha\alpha | \alpha \in \{a, b\}^*\}$ is not context-free.

Lemma (Ogden's Lemma). *Let L be a CFL. Then $\exists n$ such that $\forall z \in L, |z| \geq n, \exists u, v, w, x, y \in \Sigma^*, z = uvwxy$, if $\geq n$ positions in z are marked as distinguished,*

- vw has at most n distinguished positions.
- vx has at least 1 distinguished position.
- $\forall i \geq 0, uv^iwx^iy \in L$.

Ogden's Lemma provides an stronger form of the pumping lemma that can be used to prove non-context free-ness.

Closure of Context-Free Languages

We have, similarly to regular languages, certain operations that, when applied to CFLs, the resulting language is also a CFL.

Definition (Substitution). Suppose terminals i and CFLs L_i . Consider a mapping $s : \Sigma \rightarrow \mathcal{P}(\Gamma^*)$ such that $s(i) = L_i$, that is further defined for strings by principle of induction:

1. *Base Case:* $s(\epsilon) = \{\epsilon\}$
2. *Inductive Case:* $s(\omega a) = s(\omega) \cdot s(a), \forall a \in \Sigma, \omega \in \Sigma^*$

s is called a *substitution*. Intuitively, we replace each symbol in the strings of one language by an entire language.

Theorem. *Given a CFL L over Σ and s a substitution on Σ such that $s(a)$ is a CFL for each $a \in \Sigma$, $s(L)$ is a CFL.*

The principle of the proof is to consider replacing each terminal in the original grammar with a nonterminal that acts as the start symbol of the substituted CFL. We then have a resultant CFG that generates $s(L)$.

Given CFLs L, L_1, L_2 and regular language R :

- $L_1 \cup L_2$ is context-free.
- $L_1 \cdot L_2$ is context-free.
- L^* is context-free.
- L^+ is context-free.
- $h(L)$ is context-free, for some homomorphism h .
- L^R is context-free. Consider the reverse of all productions.
- $L \cap R$ is context-free. Consider the PDA for L , and DFA for R . We can simulate the DFA with the analogue for parallelisation of DFAs.
- $h^{-1}(L)$ is context-free, for some homomorphism h .

Remark. $\bar{L}$ and $L_1 - L_2$ are not necessarily context-free.

Remark. CFLs are not closed under intersection. Consider the CFLs $L_1 = \{0^n 1^n 2^i | n \geq 1, i \geq 1\}$ and $L_2 = \{0^i 1^n 2^n | n \geq 1, i \geq 1\}$. We have the intersection $L = \{0^n 1^n 2^n | n \geq 1\}$ which is not context-free.

Decision Problems of CFLs

We may examine some decision problems of CFLs. However, in CFLs, we have to take into account the complexities of converting between the various forms of representation, that is required for their decision problems.

We have the linear time conversions from a CFG to a PDA, from a PDA accepting by final state to a PDA accepting by empty stack and vice-versa. On the other hand, the conversion from a PDA to a CFG is requires an upper bound of $O(n^3)$ on the constructed grammar's length and hence running time. Additionally, to convert a general CFG to its CNF requires $O(n^2)$ for the length of the resultant grammar, and running time to construct the unit pairs and eliminate unit productions.

Given these conversions, we are still highly limited in what we may decide about a CFL, many of which is undecidable as shown in a later chapter.

We may check the **emptiness of CFLs** by running the algorithm to find all generating symbols in its CFG and checking whether the start symbol S is generating. That is,

$$L = \emptyset \iff S \text{ is not generating}$$

We may decide the **membership of a string** $\omega = a_1 a_2 \cdots a_n$ in a CFL L . This can be done with dynamic programming, using the CYK algorithm using a CNF grammar for the language. We construct a triangular table where each entry X_{ij} is the set of variables A such that $A \Rightarrow^* a_i a_{i+1} \cdots a_j$.

X_{15}				
X_{14}	X_{25}			
X_{13}	X_{24}	X_{35}		
X_{12}	X_{23}	X_{34}	X_{45}	
X_{11}	X_{22}	X_{33}	X_{44}	X_{55}
a_1	a_2	a_3	a_4	a_5

Table constructed by CYK Algorithm for $n = 5$

We compute the rows bottom-up, with the following inductive algorithm:

1. *Base Case:* Each X_{ii} is the set of nonterminals which generate a_i
2. *Inductive Step:* $X_{i,j}$ contains all A such that $A \rightarrow BC$, where $B \in X_{i,k}$ and $C \in X_{k+1,j}$ for some $i \leq k < j$.

We then have the membership of the string if the start symbol generates the full length of the string. That is,

$$\omega \in L \iff S \in X_{1,n}$$

Remark. The running time of the CYK algorithm is $O(n^3)$, in comparison to naive recursive searches for membership of strings which require exponential time.

Remark. The following CFL decision problems are undecidable:

- Is a given CFG G ambiguous?

- Is a given CFL inherently ambiguous?
- Is the intersection of two CFLs empty?
- Are two CFLs the same?
- Is a given CFL equal to Σ^* ?
- Is a given CFL a regular language?

3.7 Deterministic PDA

PDAs are nondeterministic by definition. However, parsers generally behave deterministically, and not unlike a deterministic PDA. We investigate, in this section, the behaviour and types of languages accepted by deterministic PDAs.

Definition (DPDA). An PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is a *DPDA* if:

1. $\forall q \in Q, a \in \{\epsilon\} \cup \Sigma$, and $X \in \Gamma$, $|\delta(q, a, X)| \leq 1$. There is only one possible move, if any, for any input and ID.
2. If $\delta(q, \epsilon, X) \neq \emptyset \implies \forall a \in \Sigma, \delta(q, a, X) = \emptyset$. That is, we do not have a choice between taking a input symbol or making a move on ϵ .

Languages Accepted by DPDAs

We define the languages accepted by DPDAs similarly to that of PDAs.

Theorem (Languages Accepted by Final State). *The languages accepted by DPDAs by final state properly include the regular languages, but are properly included in the CFLs.*

On the other hand, DPDAs that accept by empty stack are highly limited, unable to accept even certain finite languages such as $\{a, aa\}$.

Definition (Prefix Property). A language L has the *prefix property* if $\forall (x, y) \in L^2$, x is not a prefix of y .

Theorem (Languages Accepted by Empty Stack). *A language L is $L(P)$ by empty stack for some DPDA $P \iff L$ has the prefix property L is $L(P')$ by final state for some DPDA P .*

Unambiguous Grammars

Theorem. *If $L = L(P)$ by acceptance through empty stack for some DPDA P , L has an unambiguous context-free grammar.*

The typical conversion of CFG from PDA (with nonterminals encoding state transition and symbol popped) yields an unambiguous context-free grammar. Intuitively, this is because there is a unique sequence of moves in the DPDA, and hence there is only one choice of production every step of the leftmost derivation of each string. There is also only one final state that can be reached due to the determinism, and hence only one of the initial branches of S will be consistent with the PDA.

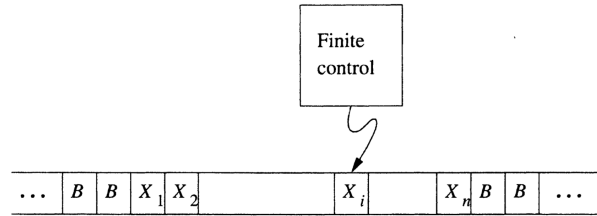
Theorem. *If $L = L(P)$ by acceptance through final state for some DPDA P , L has an unambiguous context-free grammar.*

This can be shown by adding an end marker $\$ \notin \Sigma$ to every string in L , then creating the context free grammar as though for acceptance through empty stack, and then introducing a production $\$ \rightarrow \epsilon$.

4 Turing Machines

Definition (Turing Machine). A TM $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where:

- Q : A finite set of states
- Σ : A finite set of input symbols. $\Sigma \subseteq \Gamma$
- Γ : A finite set of tape symbols
- δ : A transition function taking as argument a state and stack symbol and returns a state, stack symbol to write and direction to move. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- q_0 : A start state, $q_0 \in Q$
- B : A blank symbol. $B \in \Gamma - \Sigma$
- F : A set of final states. $F \in \mathcal{P}(Q)$



A TM consists of a finite control, which can be in any of a finite set of states, a infinite tape divided into cells, each holding one of a finite set of symbols.

Initially, the input is written on the tape with blank symbols infinitely to the left and right of the input. The tape head is positioned on the leftmost cell holding an input symbol.

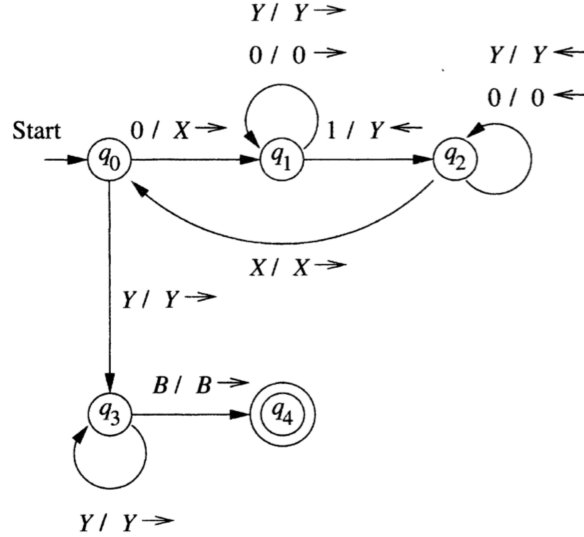
The transition function δ encapsulates a move of the turing machine, consisting:

1. A change in state in the control.
2. A replacement of the tape symbol in the read cell.
3. A move to the left or right of the current cell.

We have alternative notations for the descriptions of TMs, similarly to DFAs and PDAs, the transition diagram and table.

State	Symbol				
	0	1	X	Y	B
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, R)
q_4	—	—	—	—	—

We use the transition table to show the moves of the TM, depending on the current state and tape symbols. The transition diagram of a TM consists of a set of nodes corresponding to the states of the TM, within which each arc from state q to p is labelled by some X/YD , where $X, Y \in \Gamma$ and $D \in \{L, R\}$. Additionally, we denote accepting states by double circles and the start state with an arrow labelled 'start' entering that state.



Definition (Instantaneous Descriptions). The **ID** of a TM, analogous to that of the PDA, is the string

$$X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n.$$

where q is the state of the TM, the tape head is scanning the symbol X_i and $X_1 X_2 \cdots X_n$ is the portion of the tape from the leftmost to the rightmost nonblank symbols, except if the head is past the ends, in which case we extend to show the blank symbols.

Notation ($\vdash$). If $\delta(q, X_i) = (p, Y, L)$, we denote

$$X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n \vdash X_1 X_2 \cdots X_{i-2} q X_{i-1} Y X_{i+1} \cdots X_n.$$

This represents one move of the TM.

Notation ($\vdash^*$). As usual, we denote $\vdash^*$ to represent zero or more moves of the TM.

4.1 Turing Machines

Acceptance by a TM

Definition (Language of a TM). The *language* of a TM M is denoted $L(M)$ and is defined by

$$L(M) = \{x \mid \exists q_f \in F, q_0 x \vdash^* \alpha q_f \beta\}.$$

That is, it is the set of strings that, given as input, would eventually transit M into an accepting state.

Definition (Halting). We say a TM halts if it enters a state q , scanning a tape symbol X , and there is no move defined. $\delta(q, X)$ is undefined.

Remark. By un-defining $\delta(q_f, X)$ for all final states q_f and tape symbols X , without changing the language accepted, the TM halts if it accepts. We can hence assume that a TM always halts when it accepts a string. Since language acceptance is equivalent to function computation, we further define the notion of function computation on a TM. Provided an input x , we interpret the content of the tape of the TM if/after it halts as the output $f(x)$.

Models Equivalent to TMs

We present certain modifications or improvements to TMs to aid in their specifications. These modifications do not extend the class of languages accepted, or extend the basic model of the TM, but only affect running times and space complexities.

Storage in Finite Control The finite control not only to represent a position in the “program” of the Turing machine, but also to hold a finite amount of data. The finite control becomes a tuple of the current state, and the stored values and variables.

Subroutines TMs can be thought of as a collection of “subroutines”, sets of states that perform some useful process. Each subroutine includes a start state, and a “return” state. As such, we may define general subroutines that allow us to “jump” to a specific return state depending on the current start state.

Multiple Tapes The tape of a TM composed of multiple “tracks”. Each symbol becomes a tuple of symbols and the tape alphabet consists of tuples of the original tape alphabet. This TM is hence capable of simulating multiple tapes.

Nondeterministic TMs The transition function may instead return a set of triples, from which the NTM can choose any of the triples to be the next move. The language accepted is hence defined as the strings such that, given as input, there exist a sequence of moves that lead to an ID with an accepting state.

Theorem. *If M_N is a nondeterministic TM, there exists a deterministic TM M_D such that $L(M_N) = L(M_D)$.*

Intuitively, in M_D , we explore the IDs of M_N “breadth-first”. That is, we find all IDs reachable in 0 moves, then all IDs reachable in 1 move, then all IDs reachable in 2 moves, as so on. This is done with a tape operating as a queue of IDs to simulate a single move of.

We may further consider several restrictions of TMs, that do not affect the capabilities of TMs.

Semi-Infinite Tapes A TM that is infinite only in one direction (WLOG, to the right) can simulate a TM that is infinite in both directions. Analogue to the simulation of multiple tapes, we allow each symbol to be a tuple of tape symbols, with each i -th cell from the origin be (x_{-i}, x_i) , where x_i is the i -th cell from the origin in the original TM model.

No Blank Symbols Write The TM is not allowed to write a blank symbol. This, coupled with semi-infinite tapes, ensure that the tape is at all times a prefix of nonblank symbols followed by an infinity of blanks. We may achieve this by creating a new tape symbol B' that effectively functions as the blank symbol.

Two Stack PDAs A PDA equipped with two stacks is as powerful as a TM. We take one stack to simulate the tape symbols to the left of the head, and the other to simulate the tape symbols to its right. Each move transits the PDA’s state, pops and reads from one stack, and pushes the corresponding output to the other.

4.2 Languages of a TM

Following specifications of TMs and the languages that they accept, we have the following for the language classes defined by TMs’ capabilities.

Recursive Enumerable Language

We first consider the languages that can be accepted by TMs.

Definition (Recursively Enumerable Languages). If L is $L(M)$ for some TM M , then L is a *recursively enumerable* (computably enumerable) language.

Codings We may define, for a string x over $\{0, 1\}^*$, its coding to be

$$i = (1x)_2 - 1.$$

Notation. ω_i denotes the string with code i .

Further we may define a coding for a TM $M = (Q, \Sigma, \Gamma, \delta, q_1, B, \{q_2\})$ where

- $Q = \{q_1, q_2, \dots, q_n\}$. If there are multiple accepting states, we may define new tape symbols and modify the transition function to compress into a single accepting state.
- $\Sigma = \{0, 1\}$, WLOG.
- $\Gamma = \{X_1, X_2, \dots, X_s\}$, where $X_1 = 0, X_2 = 1, X_3 = B$.
- The directions $L = D_1$ and $R = D_2$.

We have, in this way, a coding for any transition

$$\delta(q_i, X_j) = (q_k, X_l, D_m) \quad \rightarrow \quad 0^i 10^j 10^k 10^l 10^m \quad i, j, k, l, m \in \mathbb{N}^*.$$

This allows for a coding for M to be $C_1 11 C_2 11 C_3 \dots C_n$ where C_i is the coding of the i -th transition of the TM.

Notation. M_i denotes the TM with code i , and W_i denotes the language accepted by the TM of code i .

Hence, we have a function from the valid binary strings to the set of all TMs, or the class of all RE languages.

Remark. Not all binary strings is a valid code for a TM or a RE language. By convention, we take M_i to be the TM with one state and no transitions, and immediately halts on any input.

Example (Diagonalisation Language). L_d is not RE, where the diagonalisation language $L_d = \{\omega_i | \omega_i \notin L(M_i)\}$ is the set of strings ω such that the TM whose code is ω does not accept ω as input.

Proof. Suppose a TM M_j .

1. Case 1: $\omega_j \in L(M_j)$. $\omega_j \notin L_d$ by definition.
2. Case 2: $\omega_j \notin L(M_j)$. $\omega_j \in L_d$ by definition.

In all cases, $L(M_j) \neq L_d$. Since M_j is an arbitrary TM, all TMs do not accept L_d , and it is not RE. ■

Example (Universal Language). The universal language $L_U = \{(M, \omega) | M \text{ accepts } \omega\}$ is RE.

Proof. Consider a TM which has four tapes: input tape (with codings of M and ω), tape to simulate M (with ω), tape for state of M , scratch tape. We may then simulate a step of M by reading the state of M , the current cell of M , and finding the corresponding transition on the input tape. We hence transit the state of M , and write into the current cell of M . By construction, the universal TM accepts L_U and L_U is RE. ■

Recursive Languages

Within the RE languages, we may further define classes of language in which a TM is able to tell us that a given string is not in the language. That is, it halts for all possible input strings.

Definition (Recursive Languages). If L is $L(M)$ for some TM M , and M halts on all inputs $\omega \in \Sigma^*$, then L is a *recursive (decidable) language*.

Theorem. If L is recursive, then $\overline{L}$ is recursive.

Proof. Suppose $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ accepts L and halts on all inputs. Consider a TM $\overline{M}$ such that

- The accepting states of M are the nonaccepting states of $\overline{M}$ with no transitions. $\overline{M}$ will halt without accepting.
- $\overline{M}$ has a new accepting state r with no transitions. $\overline{M}$ will halt and accept.
- For each nonaccepting state $q \in Q - F$ and tape symbol $\alpha \in \Gamma$, if $\delta(q, \alpha)$ is not defined, $\bar{\delta}(q, \alpha) = r$.
- All other transitions from nonaccepting states are unchanged. $q \in Q - F$, $\alpha \in \Gamma$, $p \in Q$, if $\delta(q, \alpha) = p$, $\bar{\delta}(q, \alpha) = p$.

Since M is guaranteed to halt, $\overline{M}$ is also guaranteed to halt, and accepts exactly strings that M does not accept.

Theorem. *A language L is recursive $\iff L$ and $\overline{L}$ is RE.*

Proof. ($\implies$) If L is recursive, $\overline{L}$ is recursive, and they both are RE.

($\impliedby$) Suppose TMs M_1, M_2 such that $L(M_1) = L$ and $L(M_2) = \overline{L}$. Consider a 2-tape TM M that simulates M_1 and M_2 in parallel. If ω is given as input and is in L , M_1 halts and accepts, and M accepts. If ω is not in L , M_2 halts and accepts, and M rejects. In all cases, one of the TMs are guaranteed to halt and accept. Therefore, M is guaranteed to halt and L is recursive.

With the two theorems, we conclude that given a language L , we have one of four cases:

- Both L and $\overline{L}$ are recursive.
- Neither L and $\overline{L}$ is RE.
- L is RE but not recursive, and $\overline{L}$ is not RE.
- $\overline{L}$ is RE but not recursive, and L is not RE.

Example (Complement of the Universal Language). The complement of the universal language $\overline{L_u}$ is not RE. Suppose that some TM M accepts $\overline{L_u}$. We can then construct a machine M' for accepting L_d :
 $M'(\omega_i)$:

1. Extract i from ω_i .
2. Run M on (M_i, ω_i) .
3. M' accepts $\iff M$ accepts.

Then L_d is RE. A contradiction. Hence $\overline{L_u}$ is not RE. ■

4.3 Undecidability

The recursive languages corresponds to what is normally defined as an algorithm, as a TM that is not guaranteed to halt may not give us enough information to conclude that a string is not in the language. We are hence unable to fully ‘solve’ the problem.

Theorem (Church-Turing Thesis). *Every effectively computable function can be computed by a Turing Machine.*

The thesis is not a theorem as effectively calculable does not have a mathematical definition. In any formalisation of computability, we have found all models of computation to be equivalent to, or strictly weaker than, a Turing Machine.

Definition (Partial Recursive Functions). If f is a function computed by some turing machine, then f is *partial recursive* (*partially computable*).

Definition (Recursive Functions). If f is a function computed by some turing machine and f is defined on all Σ^* , then f is *recursive* (*computable*).

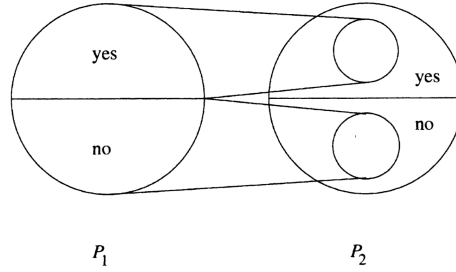
Since function computation is equivalent to language acceptance, by the Church-Turing Thesis, the class of recursive languages are intuitively the ‘limits of computation’ of traditional computers.

Reductions

Definition (Turing Reducibility). We say that a problem P_1 is reducible to P_2 , $P_1 \leq_m P_2$, if there exists a recursive function f such that

$$x \in P_1 \iff f(x) \in P_2.$$

That is, we can convert each instance of problem P_1 to some problem in P_2 that has the same answer. P_2 is as hard as or harder than P_1 .



Theorem (Reductions). Suppose $P_1 \leq_m P_2$. Then

1. If P_1 is not recursive, P_2 is not recursive.
2. If P_1 is not RE, P_2 is not RE.
3. If P_2 is recursive, P_1 is recursive.
4. If P_2 is RE, P_1 is RE.

To show that a problem P is hard, we pick a problem P' which is hard and then show that $P' \leq_m P$, by constructing a recursive function f such that $x \in P' \iff f(x) \in P$.

Example (TMs with an Nonempty Language). The set of encoded TMs with at least one string, $L_{ne} = \{M \mid L(M) \neq \emptyset\}$ is RE but not recursive.

Proof. Consider a TM M' , such that:

```

M'(M):
  for t = 0 to ∞:
    for i = 0 to t:
      if M(ωi) accepts within t steps, M' accepts.
    endfor
  endfor
endM'

```

The algorithm tries ω_1 with 1 step, then ω_1, ω_2 with 2 steps, and so on. If M accepts any string in the alphabet, it will halt and accept. Hence $L(M) = L_{ne}$ and L_{ne} is RE.

To show that L_{ne} is not recursive, we consider the complement of L_{ne} , $L_e = \{M \mid L(M) = \emptyset\}$, and show that $\overline{L_U} \leq_m L_e$, and hence L_e is not RE. We want to find a recursive function that finds a TM M' such that

$$L(M') = \emptyset \iff \omega \notin L(M).$$

Given input (M, ω) , we define

```

M'(x):
  for t = 0 to ∞:
    if M(ω) accepts within t steps, M' accepts.
  endfor
endM'

```

If M accepts ω , M' accepts all strings and $L(M') \neq \emptyset$. If M does not accept ω , M' does not accept any output. Since $\overline{L_U}$ is not RE, L_e is not RE.

L_{ne} is RE but L_e is not RE, hence they are not recursive.

Properties of RE Languages

Definition (Properties). A *property* of the RE languages is a subset of the RE languages.

Definition (Trivial Properties). A *trivial property* is a property that is either empty or all RE languages. A property is nontrivial $\iff$ there exists a language that fulfils a property and a language that doesn't.

Definition (Semantic Properties). A property P is semantic if it pertains only the behaviour of the program, rather than its syntax.

$$L_1 = L_2 \implies L_1 \in P \iff L_2 \in P$$

Theorem (Rice Theorem). All nontrivial semantic properties of the RE languages is undecidable.

Proof. Suppose $\mathcal{P}$ a nontrivial property of RE languages. We show that $L_e \leq L_{\mathcal{P}}$. That is, there exists a recursive function f such that

$$M \in L_e \iff f(M) \in L_{\mathcal{P}}.$$

WLOG, assume $\emptyset$ satisfies $\mathcal{P}$ (otherwise, consider $\overline{\mathcal{P}}$). Let L be a RE language that does not satisfy $\mathcal{P}$, and M'' the machine that accepts L . Given a TM M , we define M' :

```

M'(x):
  for t = 0 to ∞:
    for i = 0 to t:
      if M(ωi) accepts within t steps ∧ M''(x) accepts within t steps, M' accepts x.
    endfor
  endfor
endM'

```

If $L(M) = \emptyset$, $L(M') = \emptyset$ that satisfies $\mathcal{P}$. If $L(M) \neq \emptyset$, $L(M') = L$ that does not satisfy $\mathcal{P}$. Since L_e is not recursive, $L_{\mathcal{P}}$ is not recursive.

Remark. Rice's theorem does not imply that everything about a TM is undecidable. For instance, syntactic properties, such as the number of states of the TM, is decidable.

4.4 Undecidable Problems

With the groundwork of undecidability and TMs laid out, we may move on to prove the undecidability of problems involving beyond TMs and their languages. We begin with the Post's Correspondence Problem, which involves strings rather than TMs.

Post's Correspondence Problem

PCP consists two lists of strings, of equal length $k \in \mathbb{N}$, over an alphabet Σ .

$$A = w_1, w_2, \dots, w_k \quad B = x_1, x_2, \dots, x_k$$

Each $i \in \mathbb{N}_{\leq k}$ specifies a *corresponding pair* (w_i, x_i) . A solution to PCP is a nontrivial sequence of integers $i_1, i_2, \dots, i_m$ taken as indices of corresponding pairs, that yield the same string when concatenated.

$$w_{i_1} w_{i_2} \cdots w_{i_m} = x_{i_1} x_{i_2} \cdots x_{i_m}.$$

PCP is the decision problem for the existence of a solution.

“Modified” PCP To prove the undecidability of PCP, we first consider a modified problem as an intermediate reduction. We define MPCP to be an instance of PCP, with the additional constraint that the first corresponding pair must also be the first pair in the solution. A solution similarly comprises a sequence of integers $i_1, i_2, \dots, i_m$ such that

$$w_1 w_{i_1} w_{i_2} \cdots w_{i_m} = x_1 x_{i_1} x_{i_2} \cdots x_{i_m}.$$

Proposition. *MPCP is reducible to PCP.*

Suppose an instance of MPCP with lists $A = w_1, w_2, \dots, w_k$ and $B = x_1, x_2, \dots, x_k$. WLOG, assume $*, \$ \notin \Sigma$. We construct a PCP instance with $C = y_0, y_1, \dots, y_{k+1}$ and $D = z_0, z_1, \dots, z_{k+1}$ where:

- For $i = 1, 2, \dots, k$, $y_i := w_i \cdot *$ and $z_i := * \cdot x_i$
- $y_0 := * \cdot y_1$ and $z_0 := z_1$
- $y_{k+1} := \$$ and $z_{k+1} := * \$$

Proof. ($\implies$) Suppose $i_1, i_2, \dots, i_m$ is a solution to the MPCP instance.

$$w_1 w_{i_1} w_{i_2} \cdots w_{i_m} = x_1 x_{i_1} x_{i_2} \cdots x_{i_m}.$$

Applying the defined transformations, we have the equal strings

$$* \cdot y_1 y_{i_1} y_{i_2} \cdots y_{i_m} = z_1 z_{i_1} z_{i_2} \cdots z_{i_m} \cdot *.$$

Since $y_0 = * \cdot y_1$, $z_0 = z_1$, and $y_{k+1} = \$$, $z_{k+1} = * \$$, we have

$$y_0 y_{i_1} y_{i_2} \cdots y_{i_m} y_{k+1} = z_0 z_{i_1} z_{i_2} \cdots z_{i_m} z_{k+1}$$

and $0, i_1, i_2, \dots, i_m, k+1$ is a solution to the PCP instance.

($\impliedby$) Suppose a solution to the PCP instance. It must begin with index 0 and end with index $k+1$ since they are the only corresponding pairs with the same start symbol and end symbol, respectively.

$$y_0 y_{i_1} y_{i_2} \cdots y_{i_m} y_{k+1} = z_0 z_{i_1} z_{i_2} \cdots z_{i_m} z_{k+1}$$

Removing the $*$ s and the final $\$$, we have

$$w_1 w_{i_1} w_{i_2} \cdots w_{i_m} = x_1 x_{i_1} x_{i_2} \cdots x_{i_m}$$

and $1, i_1, i_2, \dots, i_m$ is a solution to the MPCP instance. ■

Proposition. *L_U is reducible to MPCP.*

Suppose a pair (M, ω) . WLOG, assume the TM never writes a blank and the tape is semi-infinite (to the right). Each ID is hence of the form $\alpha q \beta$ where $\alpha \in \Gamma^+$, $q \in Q$, $\beta \in \Gamma^*$. For $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, we define the following pairs in our MPCP instance

1. $w_1 = \#$ and $x_1 = \# q_0 w \#$ which must start the solution of MPCP to begin the simulation of M .
2. $w_2 = x_2 = \#$. For each $X \in \Gamma$, $w_i = x_i = X$.

3. For all $q \in Q - F$, $p \in Q$, $X, Y, Z \in \Gamma$,
- $w_i = qX$, $x_i = Yp$ if $\delta(q, X) = (p, Y, R)$
 - $w_i = ZqX$, $x_i = pZY$ if $\delta(q, X) = (p, Y, L)$
 - $w_i = q\#$, $x_i = Yp\#$ if $\delta(q, B) = (p, Y, R)$
 - $w_i = Zq\#$, $x_i = pZY\#$ if $\delta(q, B) = (p, Y, L)$

Each of these pairs extend the B string to add the next ID and extend the A string to match the B string.

4. For all $q \in F$, $X, Y \in \Gamma$

- $w_i = XqY$, $x_i = q$
- $w_i = Xq$, $x_i = q$
- $w_i = qY$, $x_i = q$

These pairs allow us to consume all tape symbols after achieving an accepting state. This allows us to simplify to match the final step.

5. $w_i = q\#\#$, $x_i = \#$. When the tape is emptied, we may use the final pair to complete the solution.

Intuitively, the MPCP simulates the computation of M on ω , each partial solution consisting strings that are prefixes of the sequence of IDs of M , $\#\alpha_1\#\alpha_2\#\alpha_3\#\dots$. The string from B is one ID ahead of that from A , unless M enters an accepting state, where it would be able to “catch up” to obtain a solution. Then M accepts $\omega \iff$ there is a solution to this instance of MPCP.

Since $L_U \leq_m MPCP \leq_m PCP$, by transitivity, since L_U is undecidable, PCP is undecidable.

Theorem. *PCP is undecidable.*

Ambiguous Grammars

Using PCP, we may further prove the undecidability of other problems, particularly that the ambiguity of a grammar is undecidable. Suppose an instance of PCP with lists $A = w_1, w_2, \dots, w_k$ and $B = x_1, x_2, \dots, x_k$. WLOG, suppose $a_1, a_2, \dots, a_k \notin \Sigma$. Consider the grammar, for $i \in [1, k]$

$$\begin{aligned} S &\rightarrow A|B \\ A &\rightarrow w_i A a_i | w_i a_i \\ B &\rightarrow x_i B a_i | x_i a_i \end{aligned}$$

The grammar is ambiguous $\iff L(G_A) \cap L(G_B) \neq \emptyset \iff$ The instance of PCP has a solution, where G_A is the grammar of productions involving A and that of B for G_B .

Remark. $\overline{L(G_A)}, \overline{L(G_B)}$ are CFLs.

Using the undecidability of the intersection of $L(G_A)$ and $L(G_B)$, we may prove the undecidability of various other decision problems.

Example ($L(G_1) = L(G_2)$). Take $L(G_1) = \overline{L(G_A)} \cup \overline{L(G_B)} = \overline{L(G_A) \cap L(G_B)}$ and $L(G_2) = (\Sigma \cup \{a_1, a_2, \dots\})^*$.

4.5 Unrestricted Grammars

Definition (Unrestricted Grammar). A UG $G = (N, \Sigma, S, P)$, where:

- N : A finite set of non-terminals
- Σ : A finite set of terminals, $N \cap \Sigma = \emptyset$

- S : A start symbol, $S \in V$
- P : A finite set of productions

A production, contrary to CFGs, is of the form $\alpha \rightarrow \beta$, where $\alpha \in (N \cup \Sigma)^* N (N \cup \Sigma)^*$ and $\beta \in (N \cup \Sigma)^*$.

Theorem. *The languages generated by unrestricted grammars are exactly the languages generated by TMs.*

Definition (Context Sensitive Grammar). A CSG is a UG with the additional constraint that $|\alpha| \leq |\beta|$.

Definition (Context Sensitive Languages). If L is $L(G)$ for some CSG G , then L is a *context-sensitive language*.

Example $(\{a^n b^n c^n \mid n \in \mathbb{N}^*\})$.

$$\begin{aligned} S &\rightarrow aSBC|aBC \\ CB &\rightarrow BC \\ aB &\rightarrow ab \\ bB &\rightarrow bb \\ bC &\rightarrow bc \\ cC &\rightarrow cc \end{aligned}$$

Remark. The class of context sensitive languages are exactly $NSPACE(O(n))$. They are closed under intersection, union and concatenation.

5 Complexities and Classes

Within the recursive class of languages accepted by TMs and hence our computers, we consider the time and space complexities of their computations. We so define on an TMs with multiple, but fixed, number of tapes, with finite tape alphabets. This allows us to divide the class into the *tractable* and the *intractable* problems.

Complexity Measures

In measuring the complexity of a program, we want to find a function $\Phi : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ that measures the “complexity”, or cost of a TM on some input. We denote $\Phi(i, x) = \Phi_i(x)$ for the complexity of TM M_i with the input x . We hence have the first of Blum’s complexity axioms:

$$Dom(\phi_i) = Dom(\Phi_i).$$

where ϕ is the i -th program/TM. Further, the complexity measure should be computable for practicality. We have the second of Blum’s complexity axioms:

$$\text{The set } \{i, x, B \in \mathbb{N}^3 \mid \Phi_i(x) < B\} \text{ is recursive.}$$

Definition (Blum Complexity Measure). A *Blum complexity measure* $\iff \Phi(i, x)$ is a partial recursive function that satisfies the two Blum’s complexity axioms.

Time/Space Complexity Measures

Definition (Running Time). The *running time* of a TM M on input ω , $Time_M(\omega)$, is the number of steps made by M before halting. If M does not halt on input x , $Time_M(x) = \infty$.

Definition (Time Complexity). The *time complexity* of a TM M is the function $T(n)$ that is the maximum of the running time of M on all inputs ω of length n .

$$M \text{ is } T(n) \text{ time bounded} \iff \forall \text{ inputs } \omega \text{ of length } n, Time_M(\omega) \leq T(n)$$

We may further define its analogues for space.

Definition (Space Consumption). The *space consumption* of a TM M on input ω , $Space_M(\omega)$, is the maximum number of cells “touched” by M on any of its worktapes before halting. If M does not halt on an input x , $Space_M(x) = \infty$.

Definition (Space Complexity). The *space complexity* of a TM M is the function $S(n)$ that is the maximum of the space consumption of M on all inputs ω of length n .

$$M \text{ is } S(n) \text{ space bounded} \iff \forall \text{ inputs } \omega \text{ of length } n, Space_M(\omega) \leq S(n)$$

Proposition (Existence of Arbitrary Difficult Problems). Suppose a recursive function f . We define

$g(x)$:
 Simulate M_x on input x .
 if M_x does not halt within $f(|x|)$ steps, output 0.
 else output $M_x(x) + 1$. // or anything not $M_x(x)$
 endg

g cannot be computed by any $f(n)$ time bound TM.

Proof. Suppose not. Consider the $f(n)$ time bound TM M_y that computes g . Consider $M_y(y)$. M_y must halt within $f(|y|)$ steps since it is $f(n)$ time bound. Hence $M_y(y)$ outputs $M_y(y) + 1$. A contradiction. ■

Definition (Space Constructible Functions). A function $S(n)$ is *fully space constructible* if there exists a $S(n)$ space bounded TM M such that on all inputs of length n , it uses space exactly $S(n)$.

Definition (Time Constructible Functions). A function $T(n)$ is *fully time constructible* if there exists a $T(n)$ time bounded TM M such that on all inputs of length n , it halts and uses time exactly $T(n)$.

5.1 Definitions

Definition (P). A language L is in the class P if there exists a **deterministic** TM M and a polynomial time complexity $T(n)$ such that $L = L(M)$ and M is $T(n)$ time bounded.

Definition (NP). A language L is in the class NP if there exists a **nondeterministic** TM M and a polynomial time complexity $T(n)$ such that $L = L(M)$ and M is $T(n)$ time bounded.

Definition (co-NP). A language L is in the class $co-NP$ if its complement $\bar{L}$ is in NP.

Definition (Certificate). A *certificate* (witness) is a string that certifies the answer to a computation, or membership of some string in a language. Intuitively, a certificate is the “answer” to the problem at hand.

Proposition. L is in NP $\iff$ there exists a polynomial $q(\cdot)$ and a polynomial-time bounded TM M such that

$$x \in L \iff \exists \text{ a certificate } c, |c| \leq q(|x|) \text{ } M \text{ accepts the pair } (x, c).$$

That is, we can verify a solution of the problem in polynomial time.

5.2 NP-Completeness

Definition (NP Hard). A language (problem) L is *NP hard* if for every language $L' \in NP$ there is a polynomial time reduction of L' to L .

Definition (NP Complete). A language (problem) L is *NP complete* if L is in NP, and it is NP hard. That is, for every language $L' \in NP$ there is a polynomial time reduction of L' to L .

Definition (Karp Reducibility). A problem P_1 is *Karp reducible* to P_2 , $P_1 \leq_m^p P_2$, if there exists a polynomial time computable recursive function f such that

$$x \in P_1 \iff f(x) \in P_2.$$

We say that P_1 is polynomial time, many-one reducible to P_2 .

Remark. We may further define an alternative notion of reducibility known as a *Cook reducibility*. A problem P_1 is *Cook reducible* to P_2 , $P_1 \leq_T^p P_2$, if, given an oracle for P_2 , there exists a polynomial time recursive algorithm that solves P_1 . This presents another type of NP-completeness known as Cook-completeness. In the course and in many conventions, we use the more restricted notion of Karp-completeness.

Since the reducible relation is reflexive and transitive, it is sufficient to prove that if L is NP complete, $L' \in NP$, $L \leq_m^p L'$, then L' is NP complete. We can hence prove the NP-completeness of some problem P with a known NP-complete problem P' that Karp-reduces to P and that P is NP.

Theorem. *If one of the NP complete problems is solvable in polynomial time, all the problems in NP are solvable in polynomial time. That is, $P = NP$.*

5.3 Known NP-Complete Problems

Satisfiability

Suppose a set U of variables and a collection C of clauses over U . Does there exist a satisfying truth assignment for C ?

Example (Instance of Satisfiability). Let $U = \{A, B, C, D, E, F\}$ be a set of variables. Consider the expression

$$(A \vee B \vee \neg C \vee E) \wedge (E \vee F \vee \neg A) (C \vee F) \wedge \dots$$

Is there a truth assignment to the literals such that the expression is true?

The 3-SAT problem is the Satisfiability problem, with each clause containing exactly 3 literals.

3-Dimensional Matching

Suppose 3 disjoint finite sets X, Y, Z of equal cardinality n , and a set $S \subseteq X \times Y \times Z$. Does S contain a matching (a subset $S' \subseteq S$ of cardinality n such that for any two distinct $(x_1, y_1, z_1), (x_2, y_2, z_2) \in S'$, $x_1 \neq x_2$, $y_1 \neq y_2$, and $z_1 \neq z_2$)?

Vertex Cover

Suppose a graph $G = (V, E)$ and a positive integer $K \leq |V|$. Is there a vertex cover V' of size $\leq K$?

Proof. Given a graph (V, E) , guess a $V' \subseteq V$ and verify if $|V'| \leq k$ and $\forall (v, w) \in E$, at least one of v, w is in V' . Vertex Cover is in NP.

Suppose an instance of 3SAT, with the set of variables $U = \{x_1, x_2, \dots, x_n\}$, the set of clauses $C = \{c_1, c_2, \dots, c_m\}$ where $c_i = (l_{i,1} \vee l_{i,2} \vee l_{i,3})$.

Consider a vertex cover instance $G = (V, E)$, where:

$$\begin{aligned} V &= \{u_i, w_i | 1 \leq i \leq n\} \cup \{z_{j,1}, z_{j,2}, z_{j,3} | 1 \leq j \leq m\} \\ E &= \{(u_i, w_i) | 1 \leq i \leq n\} \cup \{(z_{j,1}, z_{j,2}), (z_{j,1}, z_{j,3}), (z_{j,2}, z_{j,3}) | 1 \leq j \leq m\} \cup \{(z_{j,r}, u_i) | l_{j,r} = x_i\} \cup \{(z_{j,r}, w_i) | l_{j,r} = \neg x_i\} \\ K &= 2m + n \end{aligned}$$

Each u_i represents x_i , w_i represents $\neg x_i$, and $z_{j,r}$ represents the $l_{j,r}$. This reduction can be done in polynomial time. We want to show that the 3SAT instance has a solution $\iff$ the vertex cover instance has a solution.

In a vertex cover,

- For each $1 \leq i \leq n$, at least one of u_i, w_i is in the cover. (A pair representing a variable and its complement)
- For each $1 \leq j \leq m$, at least two of $z_{j,1}, z_{j,2}, z_{j,3}$ is in the cover. (A triangle representing each clause)

Since we have the size bound $K = 2m + n$, *exactly* one and two (respectively) of the vertices are in the cover.

($\implies$) Suppose the 3SAT instance is satisfiable. By choosing $u_i \iff x_i$ is true and $w_i \iff x_i$ is false, and choosing two of $z_{j,a}$ such that the one left out represents the true literal, we have a vertex cover of G .

($\impliedby$) Suppose the vertex cover instance has a solution. Consider the truth assignment such that x_i is true $\iff u_i$ is in the vertex cover. It can be shown that if $z_{j,r}$ is not in the vertex cover then $l_{j,r}$ must be true.

Hence Vertex Cover is NP Complete. ■

MAX-CUT

Suppose an undirected graph $G = (V, E)$ and a positive integer $K \leq |E|$. Does there exist a cut of G with size $> K$?

Clique

Suppose a graph $G = (V, E)$ and a positive integer $K \leq |V|$. Does G contain a clique of size $\geq K$ (a subset larger than K of V that can form a complete graph with edges in E)?

Independent Set

Suppose a graph $G = (V, E)$ and a positive integer $K \leq |V|$. Does G contain an independent set of size $\geq K$ (a subset larger than K of V such that no distinct pair of vertices are adjacent)?

Proof. Given a graph (V, E) , guess a $V' \subseteq V$ and verify if $|V'| \leq K$ and that it is a complete graph. Clique is in NP. Similarly for Independent Set.

Suppose $G = (V, E)$. V' is a vertex cover of $G \iff V - V'$ is an independent set of $G \iff V - V'$ is a clique of G' , where G' is the complement of G . That is, the graph has a vertex cover of size $k \iff G$ has an independent set of size $|V| - k \iff G'$ has a clique of size $|V| - k$.

Hence Clique/Independent Set is NP Complete. ■

Hamiltonian Circuit

Suppose a graph $G = (V, E)$. Does G contain a Hamiltonian circuit (a simple circuit that goes through all vertices of G)?

Partition

Suppose a finite set A and a size function $s : A \rightarrow \mathbb{R}_+^*$. Is there a subset $A' \subseteq A$ such that

$$\sum_{a \in A'} s(a) = \sum_{a \in A - A'} s(a)?$$

Set Cover

Suppose a finite set A , a collection of sets from A $\{S_1, S_2, \dots, S_m\} \subseteq \mathcal{P}(A)$, and $k \in \mathbb{N}$. Does there exist a subset $Y \subseteq \{1, 2, \dots, m\}$, $|Y| \leq k$ such that

$$A \subseteq \bigcup_{i \in Y} S_i?$$

Travelling Salesman Problem

Suppose a complete weighted graph $G = (V, E)$, and a bound $B \in \mathbb{R}_+^*$. Is there a hamiltonian circuit of total weight $\leq B$?

5.4 Modifications to TMs

Theorem (Tape Compression). *Suppose a constant $c \in \mathbb{R}_+^*$. If a language L is accepted by a $S(n)$ space bounded TM M with k tapes, there exists a TM M' with k tapes that is $\lceil cS(n) \rceil$ space bounded that accepts L .*

Proof. Suppose a TM M that is $S(n)$ space bounded and accepts a language L .

Consider the TM M' , where each cell of a worktape of M' encodes m cells of the corresponding tape of M . Then the tape alphabet $\Gamma_{M'} = \Gamma_M^m$. Through variable storage in the finite control of M' , we keep track of which of the m cells is the currently scanned cell. Hence M' is able to simulate M , and accepts $\iff M$ accepts.

The space used by M' on input x is:

$$\lceil \frac{\text{Space}_M(x)}{m} \rceil.$$

Taking $m > \frac{1}{c}$, M' is $\lceil cS(n) \rceil$ space bounded. ■

Theorem (Linear Speedup). *Suppose a constant $c \in \mathbb{R}_+^*$. Suppose a language L is accepted by a $T(n)$ time bounded TM M with $k \geq 2$ tapes, where*

$$\lim_{n \rightarrow \infty} \frac{T(n)}{n} = \infty.$$

Then there exists a TM M' with k tapes that is $\lceil cT(n) \rceil$ time bounded that accepts L .

Corollary (Linear Speedup). *Suppose a constant $c \in \mathbb{R}_+^*$. Suppose a language L is accepted by a $T(n)$ time bounded TM M with $k \geq 2$ tapes, where*

$$T(n) = dn, \quad d \in \mathbb{R}_+^*.$$

Then there exists a TM M' with k tapes that is $(1+c)n$ time bounded that accepts L .

5.5 Complexity Classes

Notation (Time and Space Complexity Classes).

$$DSPACE(S(n)) = \{L \mid \exists \text{ DTM } M, L(M) = L \wedge M \text{ is } S(n) \text{ space bounded.}\}$$

$$DTIME(T(n)) = \{L \mid \exists \text{ DTM } M, L(M) = L \wedge M \text{ is } T(n) \text{ time bounded.}\}$$

$$NSPACE(S(n)) = \{L \mid \exists \text{ NTM } M, L(M) = L \wedge M \text{ is } S(n) \text{ space bounded.}\}$$

$$NTIME(T(n)) = \{L \mid \exists \text{ NTM } M, L(M) = L \wedge M \text{ is } T(n) \text{ time bounded.}\}$$

Theorem (Relationships between Complexity Classes).

$$DTIME(S(n)) \subseteq DSPACE(S(n))$$

If $L \in DSPACE(S(n))$ and $S(n) \geq \log n$, then there exists a constant $c(L) \in \mathbb{R}_+^*$, $L \in DTIME(c^{S(n)})$

If $L \in NTIME(T(n))$, then there exists a constant $c(L) \in \mathbb{R}_+^*$, $L \in DTIME(c^{T(n)})$

Theorem (Savitch's Theorem). If $S(n) \geq \log n$, and $S(n)$ is fully space constructible, then

$$NSPACE(S(n)) \subseteq DSPACE((S(n))^2).$$

Theorem. The class of languages that can be accepted in polynomial space by a DTM $PSPACE$ is equal to the class of languages that can be accepted in polynomial space by a NTM $NPSPACE$. $PSPACE = NPSPACE$

Hierachy Theorems

Theorem. If L is accepted by a $S(n) \geq \log n$ space bounded TM, then L can be accepted by a $S(n)$ space bounded machine which halts on all inputs.

Hence, WLOG, we may assume that a TM with a large enough space bound halts on all inputs.

Theorem (Space Hierachy). Suppose $S_2(n), S_1(n) \geq \log n$. If $S_2(n)$ is fully space constructible and

$$\lim_{n \rightarrow \infty} \frac{S_1(n)}{S_2(n)} = 0,$$

there is a language in $DSPACE(S_2(n)) - DSPACE(S_1(n))$.

Proof. Consider the TM M which is $S_2(n)$ space bounded, with a fixed number of tapes.

```

M(x):
  if x is of the form  $1^k$ , reject.
  Mark out space  $S_2(|1^k 0x|)$  on the work tape.
  Simulate machine  $M_x$  on input  $1^k 0x$ .
  if  $M_x$  attempts to use more than  $S_2(|1^k 0x|)$ ,  $M$  rejects.
  if  $M_x$  halts and rejects the input,  $M$  accepts.
endM

```

Let $L = L(M)$. Since M is $S_2(n)$ space bounded, $L \in DSPACE(S_2(n))$.

Suppose by way of contradiction that some TM M' is $S_1(n)$ space bounded accepting L . There exists another 2-tape machine M_x accepting L using space $\leq dS_1(n)$ for some constant d . WLOG, assume M_x halts on all inputs.

Simulation of M_x requires space $cS_1(n)$ for some constant c . Hence, let k be large enough such that

$$cS_1(|1^k 0x|) < S_2(|1^k 0x|).$$

The simulation of M_x must complete, and M accepts $\iff M_x$ did not. M_x accepts a different language than L . A contradiction. ■

Theorem (Time Hierachy). Suppose $T_2(n), T_1(n) \geq (1 + \epsilon)n$. If $T_2(n)$ is fully time constructible and

$$\lim_{n \rightarrow \infty} \frac{T_1(n) \log(T_1(n))}{T_2(n)} = 0,$$

there is a language in $DTIME(T_2(n)) - DTIME(T_1(n))$.

Proof. Consider a machine M with at least 5 tapes (or more for fully time constructibility of $T_2(n)$). While running M , M keeps track of the number of steps taken, and rejects if $T_2(n)$ time is exceeded.

$M(x)$:

if x is of the form 1^k , reject.

Simulate M_x on input $1^k 0x$.

if M_x halts (does not exceed the time limit) and rejects the input, M accepts.

Let $L = L(M)$. Since M is $O(T_2(n))$ time bounded, $L \in DTIME(T_2(n))$ by linear speedup.

Suppose by way of contradiction that some TM M' is $T_1(n)$ time bounded accepting L . There exists another 2-tape machine M_x accepting L that is $dT_1(n) \log T_1(n)$ time bounded, for some constant d .

Simulation of M_x requires time $cT_1(n) \log T_1(n)$ for some constant c . Hence, let k be large enough such that

$$cT_1(|1^k 0x|) \log(T_1(|1^k 0x|)) < T_2(|1^k 0x|).$$

The simulation of M_x must complete, and M accepts $\iff M_x$ did not. M_x accepts a different language than L . A contradiction. ■